第2节 Milvus向量库

之前我们都是使用 `InMemoryVectorStore` 这样的测试用向量库，而在项目中就必须使用企业级的高性能向量库了。今天我们就来学习企业最常用的高性能向量库就是Milvus

学习目标：

- 能使用Milvus定义Collection的Schema约束
- 能使用Milvus定义Collection的稀疏和稠密索引
- 能使用Milvus创建Collection
- 能使用Milvus保存数据
- 能使用Milvus混合检索
- 能实现查询结果重排

1. 认识Milvus

本节我们先认识下Milvus，然后安装部署。

1.1 什么是Milvus

Milvus 由 Zilliz 开发，并即将捐赠给 Linux 基金会旗下的 LF AI & Data Foundation，现已成为全球领先的开源向量数据库项目之一。 该项目采用 Apache 2.0 许可证发布，大多数贡献者来自高性能计算（HPC）领域的专家，他们专长于构建大规模系统和优化硬件感知代码。核心贡献者包括来自 Zilliz、ARM、NVIDIA、AMD、Intel、Meta、IBM、Salesforce、阿里巴巴和微软的专业人士。

有趣的是，Zilliz 的每个开源项目都以鸟类命名，这一命名惯例象征着自由、远见以及技术的敏捷演进。Milvus是鹰科，鸢属的一种猛禽，以其飞行速度快、视力敏锐和非凡的适应能力而著称。

Milvus 提供三种部署模式，覆盖广泛的数据规模——从 Jupyter Notebook 中的本地原型设计，到管理数百亿向量的大型 Kubernetes 集群：

- Milvus Lite 是一个 Python 库，可轻松集成到您的应用程序中。作为 Milvus 的轻量级版本，它非常适合在 Jupyter Notebook 中进行快速原型设计，或在资源有限的边缘设备上运行。了解更多。
- Milvus Standalone 是一种单服务器部署方案，所有组件打包到单个 Docker 镜像中，便于部署。了解更多。
- Milvus Distributed 可部署在 Kubernetes 集群上，采用专为数十亿量级甚至更大规模场景设计的云原生架构。该架构确保了关键组件的冗余性。了解更多。

1.2 安装Milvus

本地开发建议使用Milvus Standalone模式，由于Milvus依赖一些其它服务，因此部署时不仅需要Milvus，还要把依赖服务一起部署。这就需要用到DockerCompose了。

在课前资料中有一个《deploy》目录，其中有准备好的部署材料（资料会不断更新，下载比较早的同学可以重新下载看看有没有）：



使用docker compose命令即可一键部署：

```
代码块
1   docker compose up -d
```

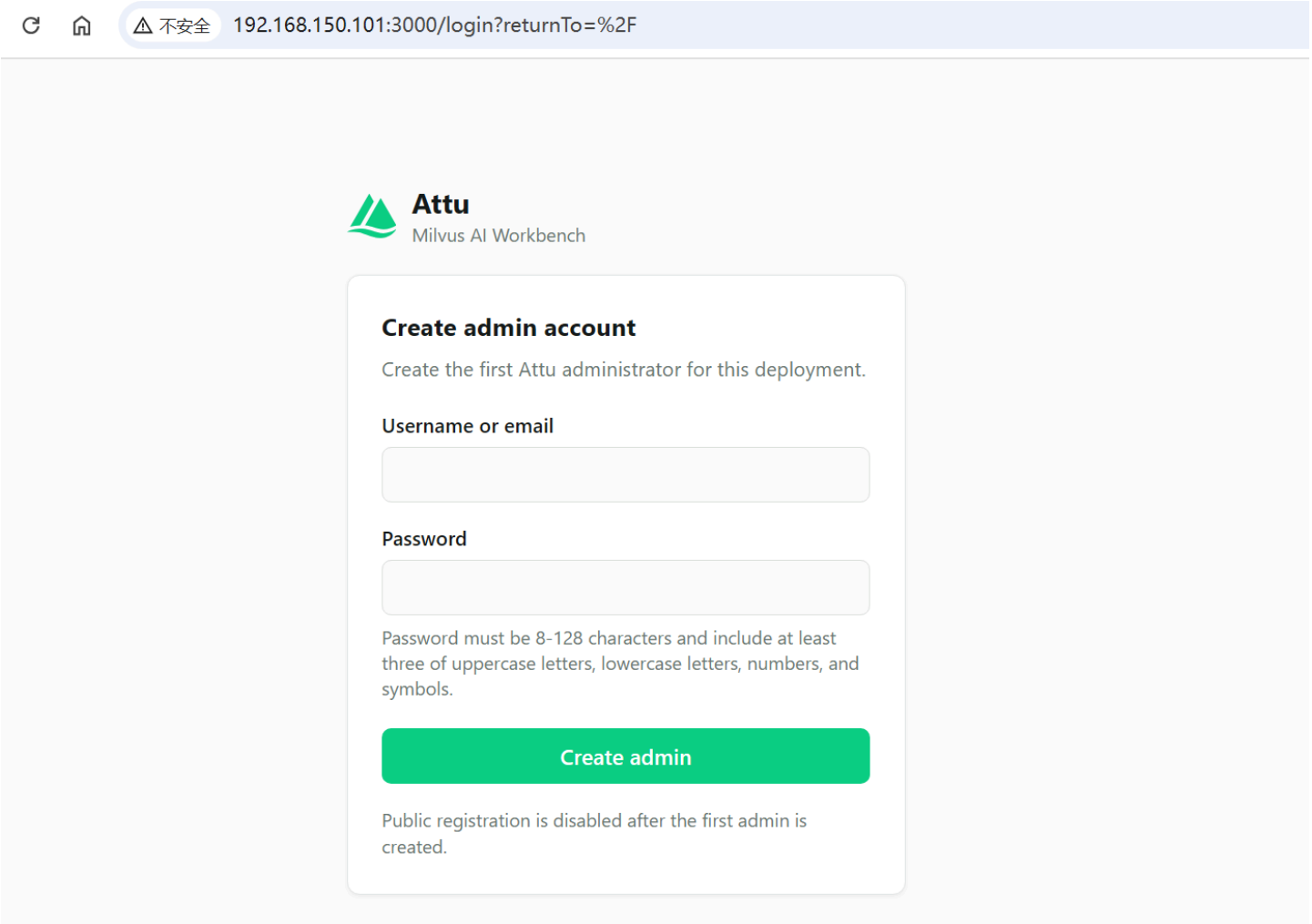
如图：

```
[root@heima deploy]#
[root@heima deploy]#
[root@heima deploy]# docker compose up -d
[+] Running 9/9
 ✔ Network deploy_heima-net      Created                                                                    0.0s
 ✔ Volume "deploy_milvus_data"   Created                                                                    0.0s
 ✔ Volume "deploy_attu-data"     Created                                                                    0.0s
 ✔ Volume "deploy_etcd_data"     Created                                                                    0.0s
 ✔ Volume "deploy_minio_data"    Created                                                                    0.0s
 ✔ Container hm-minio            Healthy                                                                   30.8s
 ✔ Container hm-etcd             Healthy                                                                   30.8s
 ✔ Container hm-milvus           Healthy                                                                   61.5s
 ✔ Container deploy-attu-1       Started                                                                   61.7s
[root@heima deploy]#
[root@heima deploy]# docker compose ps
NAME            IMAGE                                  COMMAND                  SERVICE   CREATED              STATUS
                PORTS
deploy-attu-1   zilliz/attu:v3.0.0                     "docker-entrypoint.s…"   attu      About a minute ago   Up 18 second
s               0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
hm-etcd         quay.io/coreos/etcd:v3.5.13            "etcd -advertise-cli…"   etcd      About a minute ago   Up About a m
inute (healthy)   2379-2380/tcp
hm-milvus       milvusdb/milvus:v3.0.0                 "/tini -- milvus run…"   milvus    About a minute ago   Up 49 second
s (healthy)     0.0.0.0:9091->9091/tcp, :::9091->9091/tcp, 0.0.0.0:19530->19530/tcp, :::19530->19530/tcp
hm-minio        minio/minio:RELEASE.2024-04-18T19-09-19Z   "/usr/bin/docker-ent…"   minio     About a minute ago   Up About a m
inute (healthy)   0.0.0.0:9000-9001->9000-9001/tcp, :::9000-9001->9000-9001/tcp
[root@heima deploy]#
[root@heima deploy]#
[root@heima deploy]#
```

全部启动成功

1.3 Milvus连接

Milvus有一个web控制台叫 `attu` ，端口是3000，在浏览器输入地址：http://192.168.150.101:3000 即可访问：

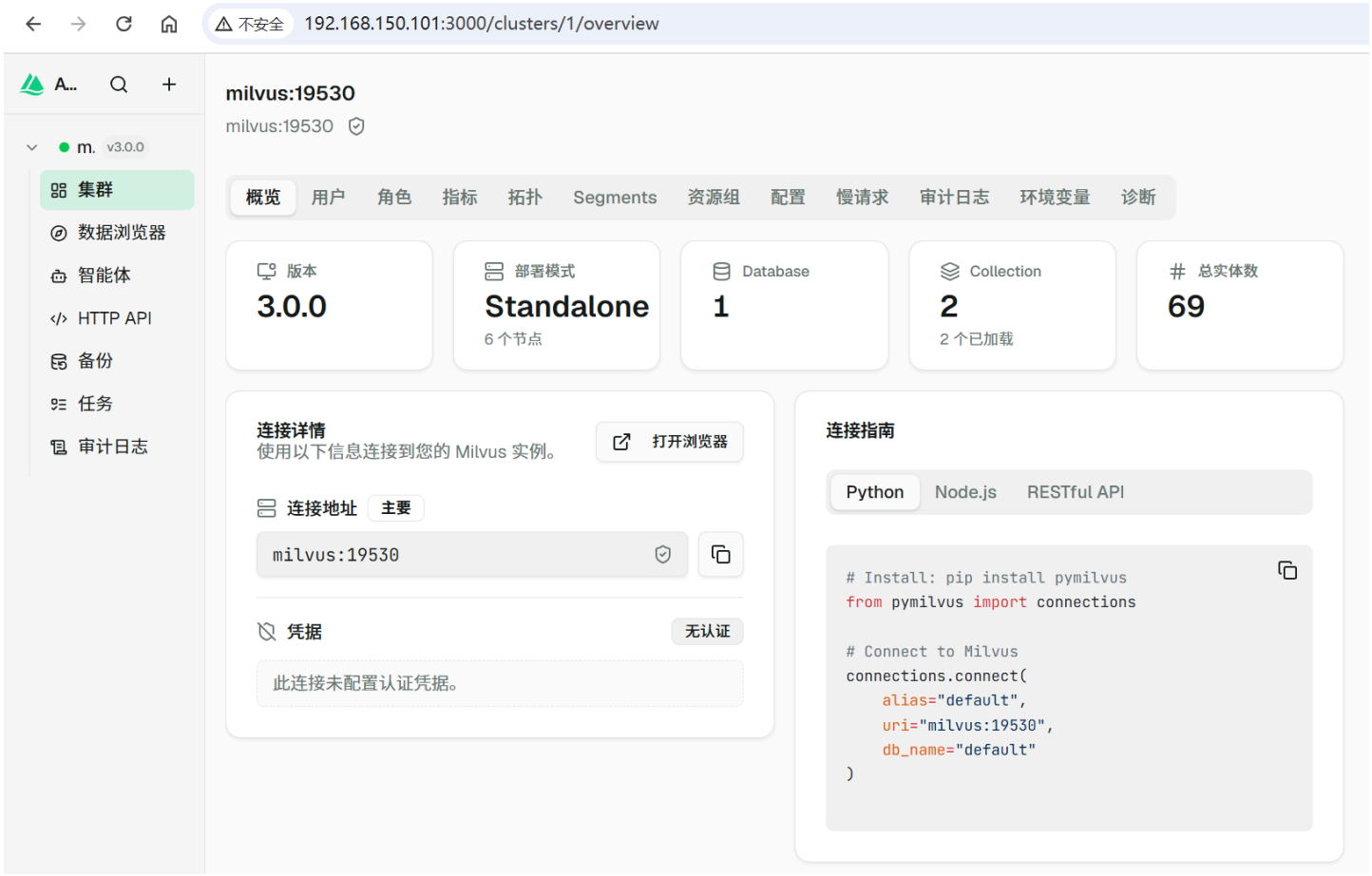


首次访问，需要注册一个管理员的用户名和密码，密码必须8位以上，且必须有大写、小写、数字、下划线中的3个类型。例如：admin/admin_123

注册后即可登录：



登录后，可以看到Milvus信息：



2. 基本概念

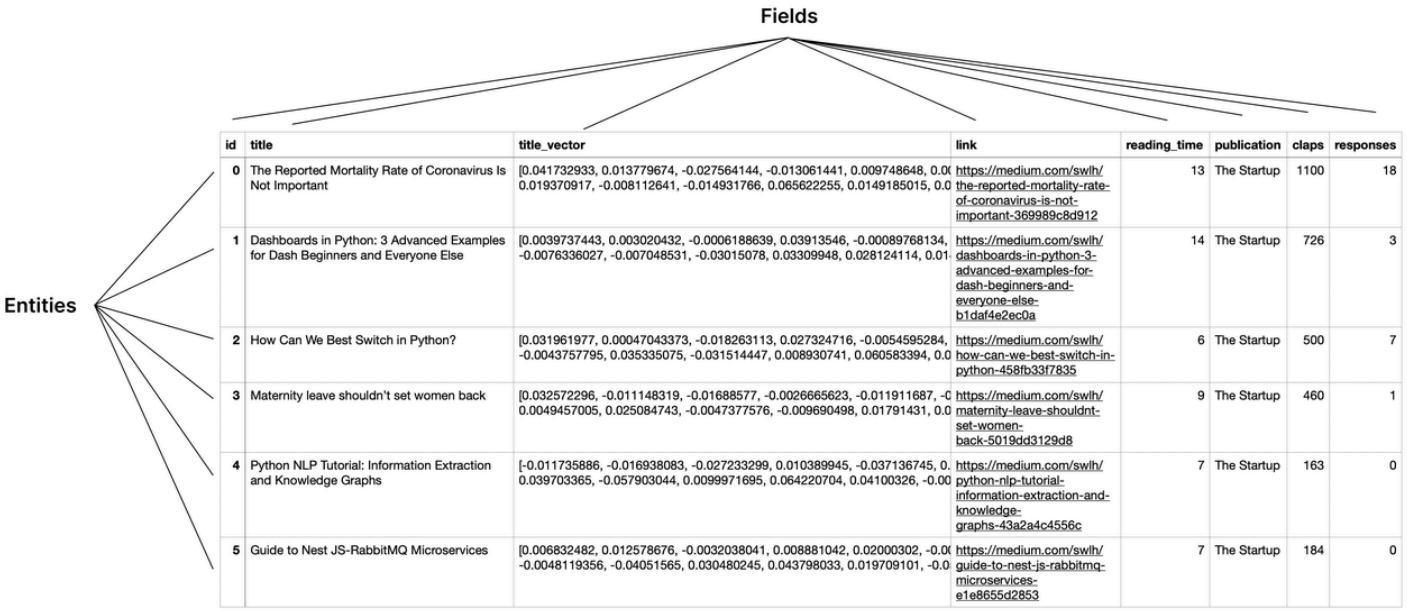
Milvus是向量数据库，与MySQL这样的关系型数据库一样都是存储数据的，因此在概念上有很多相似之处。

2.1 Collection、Field、Entity

Milvus 中比较核心的概念有 3 个。它们与 MySQL 中的概念相似，但并不完全等价：

Milvus	MySQL	含义
Collection	Table	一组结构相同的数据集合
Field	Column	描述数据的某个属性，例如标题、文本、向量
Entity	Row	Collection 中的一条完整数据

下图展示了一个包含 8 个 Field 和 6 个 Entity 的 Collection：



id	title	title_vector	link	reading_time	publication	claps	responses
0	The Reported Mortality Rate of Coronavirus Is Not Important	[0.041732933, 0.013779674, -0.027564144, -0.013061441, 0.009748648, 0.0(0.019370917, -0.008112641, -0.014931766, 0.065622255, 0.0149185015, 0.0	https://medium.com/swlh/the-reported-mortality-rate-of-coronavirus-is-not-important-369989c8d912	13	The Startup	1100	18
1	Dashboards in Python: 3 Advanced Examples for Dash Beginners and Everyone Else	[0.0039737443, 0.003020432, -0.0006188639, 0.03913546, -0.00089768134, -0.0076336027, -0.007048531, -0.03015078, 0.03309948, 0.028124114, 0.01	https://medium.com/swlh/dashboards-in-python-3-advanced-examples-for-dash-beginners-and-everyone-else-b1daf4e2ec0a	14	The Startup	726	3
2	How Can We Best Switch in Python?	[0.031961977, 0.00047043373, -0.018263113, 0.027324716, -0.0054595284, -0.0043757795, 0.035335075, -0.031514447, 0.008930741, 0.060583394, 0.0	https://medium.com/swlh/how-can-we-best-switch-in-python-458fb33f7835	6	The Startup	500	7
3	Maternity leave shouldn't set women back	[0.032572296, -0.011148319, -0.01688577, -0.0026665623, -0.011911687, -(0.0049457005, 0.025084743, -0.0047377576, -0.009690498, 0.01791431, 0.0	https://medium.com/swlh/maternity-leave-shouldnt-set-women-back-5019dd3129d8	9	The Startup	460	1
4	Python NLP Tutorial: Information Extraction and Knowledge Graphs	[-0.011735886, -0.016938083, -0.027233299, 0.010389945, -0.037136745, 0. 0.039703365, -0.057903044, 0.0099971695, 0.064220704, 0.04100326, -0.00	https://medium.com/swlh/python-nlp-tutorial-information-extraction-and-knowledge-graphs-43a2a4c4556c	7	The Startup	163	0
5	Guide to Nest JS-RabbitMQ Microservices	[0.006832482, 0.012578676, -0.0032038041, 0.008881042, 0.02000302, -0.0(-0.0048119356, -0.04051565, 0.030480245, 0.043798033, 0.019709101, -0.0	https://medium.com/swlh/guide-to-nest-js-rabbitmq-microservices-e1e8655d2853	7	The Startup	184	0

Figure 1: A Medium article collection in Zilliz Cloud database.

2.2 Schema

MySQL中需要定义表结构，Milvus中也需要定义Collection结构。在Milvus中Collection结构是由Schema来控制的。

Schema 用来定义 Collection 中包含哪些 Field，以及每个 Field 的名称、数据类型和相关属性。

常见的 Schema 属性可以按使用场景分为以下几类：

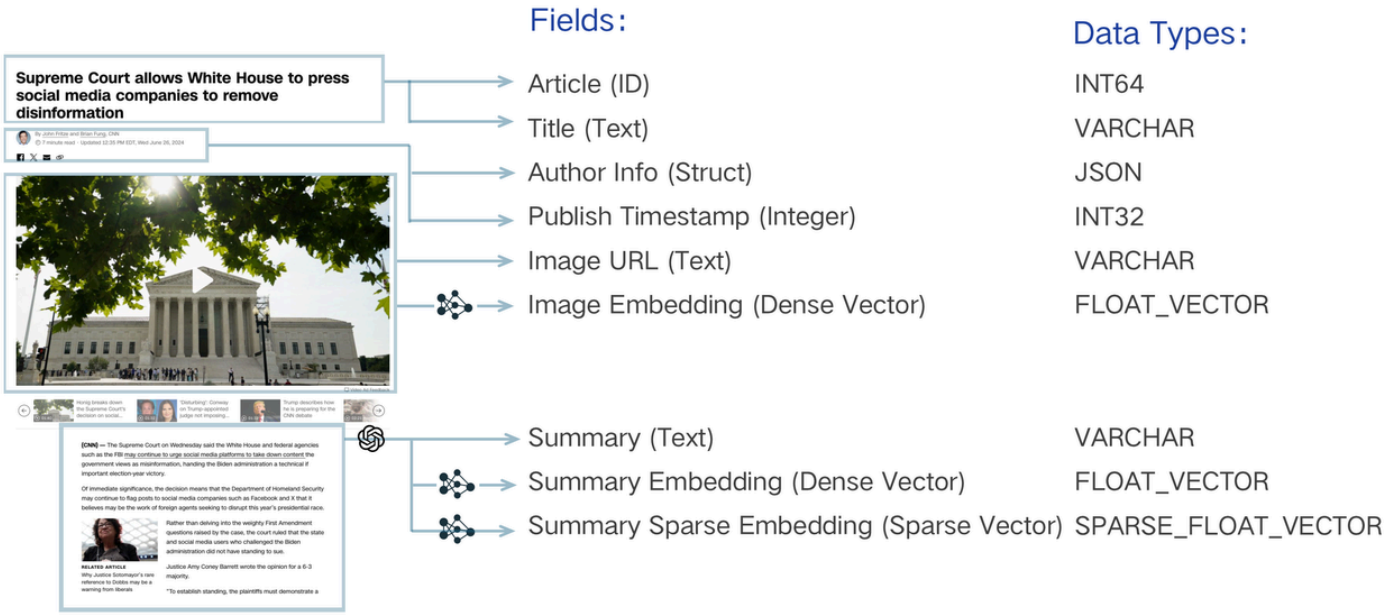
适用场景	属性	说明
通用	`field_name`	字段名称
	`datatype`	字段的数据类型
	`description`	字段说明
主键	`is_primary`	是否为主键
	`auto_id`	是否由 Milvus 自动生成主键
字符串	`max_length`	字符串字段的最大长度
向量	`dim`	向量的维度
数组	`element_type`	数组中元素的数据类型

	`max_capacity`	数组最多可以保存的元素数量

以上属性并不是每个字段都需要配置，而是根据具体的数据类型来选择，Milvus支持的数据类型有很多，常见的有：

- `VARCHAR`：也就是字符串，除了通用属性外，还需要设置`max_length`属性
- `INT32`、`INT64`：数字，通常设定通用属性即可
- `ARRAY`：数组，元素可以是其它类型，数组除了通用属性，还需要设定下列特有属性
  - `element_type`：数组中元素类型
  - `max_capacity`：数组元素数量
- 向量类型：下一节再细讲

在实际开发中，我们要根据文档的数据特点来选择合适的字段类型，例如官网给出的示例：



2.3 Index

Index，也就是索引，用来提高数据检索速度。根据字段的数据类型不同，Milvus 中常用的索引可以分为三类：

数据类型	常见索引	用途
稠密向量	`AUTOINDEX`、`HNSW`、`IVF_FLAT`	加速向量相似度检索
稀疏向量	`SPARSE_INVERTED_INDEX`	加速稀疏向量检索

标量字段（普通字段）	`INVERTED`、`BITMAP`	条件过滤，缩减向量检索范围

例如，一个保险产品文档切片：

代码块

```
{
  "product_name": "达尔文12号重大疾病保险",
  "category": "重疾险",
  "content": "等待期后，被保人首次确诊合同约定的重大疾病，保险公司按照基本保险金额给付重大疾病保险金。"
}
```

保存到 Milvus 时，这条文档可以同时包含下面几类数据：

Field	示例数据	数据类型	作用
`product_name`	达尔文12号重大疾病保险	普通字段	条件过滤
`category`	重疾险	普通字段	条件过滤
`content`	等待期后，被保险人首次确诊……	普通字段	展示
`dense_vector`	`[0.12, -0.35, 0.67, ...]`	稠密向量	向量检索
`sparse_vector`	`{1: 0.8, 3: 0.9, ...}`	稀疏向量	向量检索

其中：

- `content`：也就是文档片段的原始内容，它的作用有两个，
  - 用来生成向量，
  - 作为查询结果返回，也就是RAG检索的文档原文
- `product_name`、`category`是文档原本的字段，用于条件过滤。类似数据库的`where key = value`
- `dense_vector`、`sparse_vector`属于是衍生字段，通常是根据`content`内容向量化得来。

标量字段，也就是普通字段是类似于where条件的过滤，比较简单。

但问题来了：

> 这里的稠密向量、稀疏向量是什么意思呢？

2.3.1 稠密向量

学习RAG时，我们使用 Embedding 模型将文本转换成向量，再根据向量相似度检索语义相近的文档。这里生成的就是**稠密向量**。

```
代码块
1   [0.12, -0.35, 0.67, 0.28, ...]
```

稠密向量的维度是固定的，例如 `qwen3-embedding:0.6b` 生成的是1024维的向量。并且大部分位置都有数值，所以称为“稠密”。

稠密向量中的数值共同表示文本的语义，基于稠密向量的相似度检索就是根据**语义检索**，即使用户使用了不同的表达方式，只要含义相近，哪怕是写错字也可能检索到相关内容。

例如：

- 原始文档：`2.1.免责声明：遇到以下情形之一，不予理赔...`
- 用户问题：`什么情况下保险不能赔?`

两句话用词不同，但语义接近，稠密向量检索通常能够找到对应文档。

2.3.2 稀疏向量

如果说稠密向量是基于语义的检索，那**稀疏向量**则是基于**关键词**的检索。

稠密向量擅长理解语义，但在产品名称、疾病名称、条款编号等需要精确匹配的场景中不一定稳定。特别是一些专业词汇，一字之差意思就完全不同。

例如，知识库中有两份保险条款：

> - 文档 A：本产品保障甲状腺乳头状癌。
> - 文档 B：本产品保障甲状腺髓样癌。

用户询问：“甲状腺髓样癌是否可以赔付？”

两份文档都包含“甲状腺”和“癌”，整体语义也非常接近。稠密向量检索可能认为它们都与问题高度相关，甚至把错误的文档 A 排在前面。

稀疏向量检索则会重点关注“髓样癌”这个关键词，使包含完全匹配词语的文档 B 获得更高分。

那么问题来了：

> 什么是稀疏向量呢？它如何实现关键词检索呢？

稀疏向量通常会有一份人类语言的**词表**。词表就是系统能够识别的所有词语集合，一个词表通常有几万到几十万词语，每个词语在向量中都有一个固定位置。例如下面这个简化的词表：

0	1	2	3	4	5	6	7	8	9	...	100000
医疗险	重疾险	意外险	被保人	等待期	保险金	投保年龄	保费	免责条款	续保	...	xxx

文档转为稀疏向量，就是把文档中出现的重要词语在词表对应位置上给权重值，其他位置则是 `0`。示例文档含有：*被保人*、*等待期*、*保险金*，等词语，转换后的稀疏向量可以简化表示为：

代码块

```
[0, 0, 0, 0.9, 0.6, 0.7, 0, 0, 0, ....., 0]
```

实际系统中的词表可能包含几万甚至几十万词语，因此稀疏向量的维度通常很高。但是，一段文档只会涉及词表中的少量词语，所以绝大多数位置都是 `0`，这就是“**稀疏**”的含义。

为了节省存储空间，实际使用时不需要保存大量的 `0`，只需要记录非零位置和对应的权重：

代码块

```
{
    3: 0.9,
    4: 0.6,
    5: 0.7
}
```

比较稀疏向量相似度，就是看关键词的出现位置和权重等是否接近，所以**稀疏向量检索更擅长关键词的精确匹配检索**。例如：地名、人名、医疗词汇等专有名词或者编号。

⚡ **说明：**

稀疏向量的词条权重有两种获取方式：

- **统计式：** `BM25` 算法、 `TF/IDF` 算法等都是基于词频统计的稀疏检索算法，需要的计算资源较少，速度快
- **学习式：** `BGE-M3` 、 `SPLADE` ，是基于神经网络模型训练实现稀疏检索，需要的计算资源多（GPU），速度慢，但理解能力会更强（带有一定的语义分析）

具体可以参考资料：

- 官方文档：https://milvus.io/docs/zh/v2.6.x/sparse_vector.md

2.3.3 索引类型

由于稀疏向量、稠密向量、普通字段，在数据格式上有很大差别，因此建立索引的方式也不同。

数据类型	保存的内容	常见索引
稠密向量	文本的语义特征	`AUTOINDEX` 、 `HNSW` 、 `IVF_FLAT`
稀疏向量	关键词及其权重	`SPARSE_INVERTED_INDEX`
普通字段	名称、分类、状态等业务数据	`INVERTED` 、 `BITMAP`

实际检索时，三类索引可以配合使用：

- 先用普通字段索引过滤数据，缩小数据检索范围，
- 再用稠密向量索引匹配语义，稀疏向量索引匹配关键词。

稠密向量和稀疏向量不是替代关系，而是互补关系，实际中经常会同时使用两种检索方式，也就是**混合检索**。

⚡ **说明：**

- 在Milvus中内置了BM25的稀疏向量实现，所以等一会儿我们会直接使用基于BM25的稀疏向量来演示。
- 想要了解更多 Milvus 的索引信息，可以参考官方文档：Index Explained | Milvus Documentation

2.4 总结

Milvus 是面向向量检索的数据库。除了向量之外，它也可以保存字符串、数字、数组等普通字段，并支持对这些字段创建索引和进行条件过滤。

Milvus的概念与MySQL很相似：

- MySQL 使用 Table 组织数据，Milvus 使用 Collection；
- MySQL 表有列，也就是字段；Milvus 的Collection有Field；
- MySQL 创建表时需要定义列，Milvus 通过 Schema 定义 Field；
- MySQL 和 Milvus 都可以通过索引提高检索效率，但 Milvus 需要根据向量字段和普通字段选择不同的 Index。

所以，我们要使用Milvus，也和使用MySQL的步骤类似：

1. 定义 Collection 的字段结构，也就是 Schema； *（MySQL定义表中每一个列的约束）*
2. 配置字段索引，也就是 Index； *（MySQL的字段索引）*
3. 根据 Schema 和 Index 创建 Collection。 *（MySQL创建表）*

3. 准备工作

正式开始演示Milvus用法之前，我们要做一些准备工作：

- 安装Milvus依赖
- 准备文档
- 初始化向量模型

3.1 安装依赖

首先是安装依赖：

代码块

```
uv add pymilvus
```

3.2 准备文档

我们先把要写入索引库的文档准备好。

在Notebook中新建一个Cell，把昨天的文档处理代码准备好：

代码块

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

def prepare_docs():
    # =============1.准备文档================
    with open('./resources/评估.md', 'r', encoding='utf-8') as f:
        doc = f.read()

    # =============2.切分文档================
    # 切分依据，这里是按照三级标题
    headers_to_split_on = [
        ("#", "h1"),
        ("##", "h2"),
        ("###", "h3"),
    ]

    # 创建切分器
    markdown_splitter = MarkdownHeaderTextSplitter(
        headers_to_split_on,
        strip_headers=False  # 在正文中是否去除标题，False，不去除
    )

    # 切分文档，返回的是 list[Document]
    docs = markdown_splitter.split_text(doc)

    return docs

docs = prepare_docs()
```

切分好的文档结构是这样的：

代码块

```
{
  "id": null,
  "metadata": {
    "h1": "第一章 教育概述",
    "h2": "第一节 中外教育家及其教育思想",
```

```
      "h3": "（一）教育学萌芽时期代表作"
    },
    "page_content": "# 第一章 教育概述  \n## 第一节 中外教育家及其教育思想  \n### （一）教育学萌芽时期代表作  \n国内：《学记》，世界最早，成文于战国末期，作者是乐正克。\n国外：昆体良《雄辩术原理》/《论演说家的教育》。",
    "type": "Document"
  }
```

3.3 向量模型

我们依然使用之前演示过的`OllamaEmbedding`：

代码块

```python
from langchain_ollama import OllamaEmbeddings

# 1.初始化向量模型，可以把文档转稠密向量
embeddings = OllamaEmbeddings(
    model="qwen3-embedding:0.6b",  # 性价比高的模型
    dimensions=1024  # 可选：减少维度以节省存储
)
```

4. Collection操作

接下来，我们来演示Milvus的客户端操作collection。

创建一个Collection的操作分为3步：

- 定义Schema，也就是Field约束
- 定义Index，也就是添加索引
- 创建Collection，调用`client.create_collection`，传入
  - `collection_name`：collection名称
  - `schema`：前面定义的schema信息
  - `index_params`：前面定义的index信息

4.1 Collection结构分析

我们准备好的文档结构是这样的：

代码块

```
{
  "id": 1,
  "metadata": {
    "h1": "第一章 教育概述",
    "h2": "第一节 中外教育家及其教育思想",
    "h3": "（一）教育学萌芽时期代表作"
  },
  "page_content": "# 第一章 教育概述  \n## 第一节 中外教育家及其教育思想  \n### （一）教育学萌芽时期代表作  \n国内：《学记》，世界最早，成文于战国末期，作者是乐正克。\n国外：昆体良《雄辩术原理》/《论演说家的教育》。",
  "type": "Document"
}
```

我们来分析下文档的Schema，也就是需要的Field：

- `id`：文档唯一标识，`INT`类型，一定需要，而且是主键
- `metadata`：元数据，也就是标题信息，可以作为过滤字段，我们存储一个h2进去演示用，`VARCHAR`类型
- `page_content`：原始文档内容，`VARCHAR`类型，稀疏和稠密向量都根据它生成

除了这3个基本字段，我们还需存储稀疏和稠密向量，用于检索文档。这两个字段属于是衍生字段，可以自定义：

- `dense`：稠密向量，类型是`FLOAT_VECTOR`
- `sparse`：稀疏向量，类型是`SPARSE_FLOAT_VECTOR`

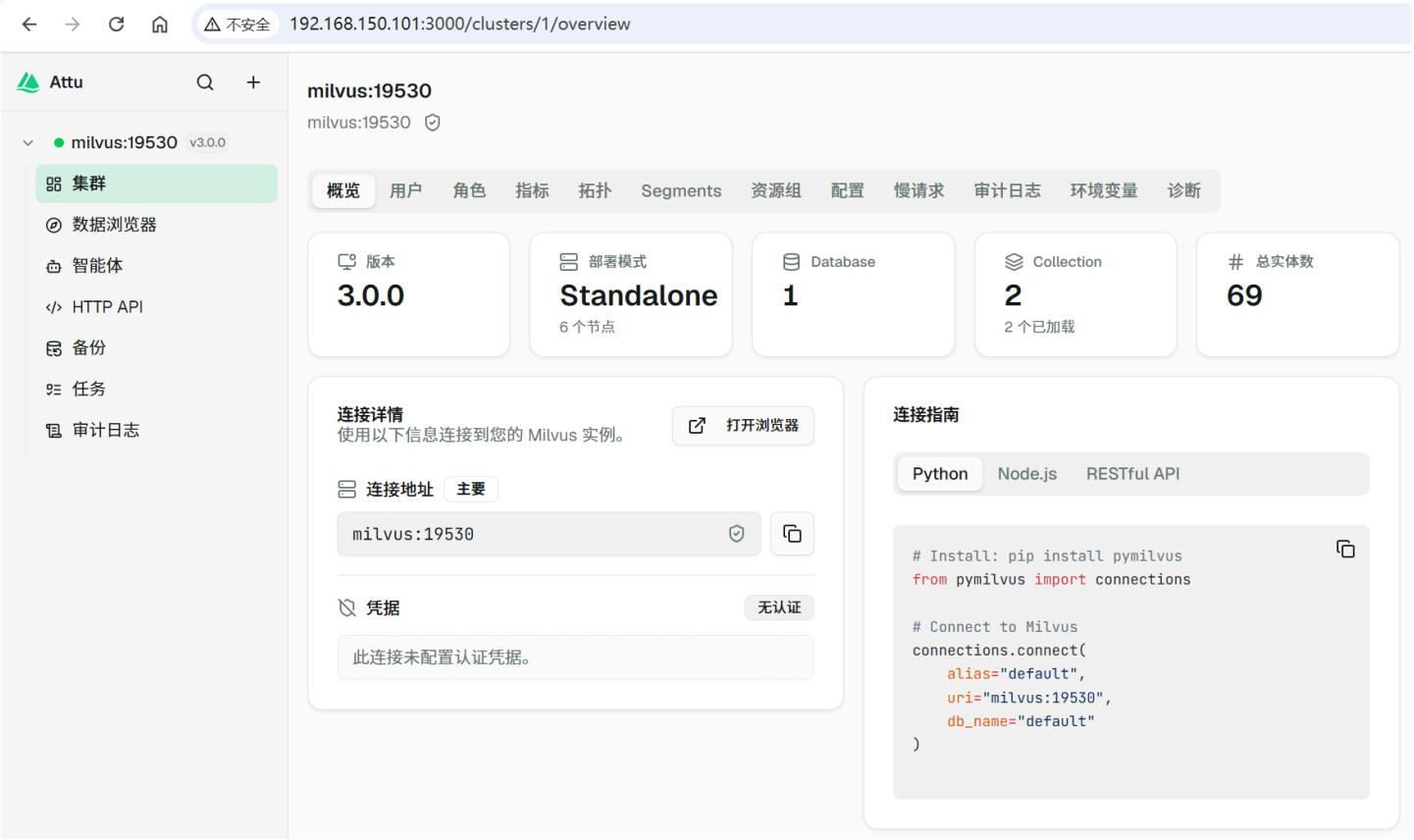
所以，最终我们向Collection中存入的数据结构应该是这样的：

代码块

```
{
    "id": 1,
    "h2": "存h2的内容作为示例",
    "content": "存page_content，也就是文档内容",
    "dense": [0.1, ...], # 存稠密向量，也就是浮点数数组，值根据content用向量模型生成
    "sparse": [0.2, ...] # 存稀疏向量，也就是浮点数数组，值根据content用BM25生成
}
```

4.2 GUI操作Collection

在之前的部署脚本中，我们已经安装了Milvus的GUI工具，访问http://192.168.150.101:3000 即可查看到控制台：



4.2.1 查看Collection

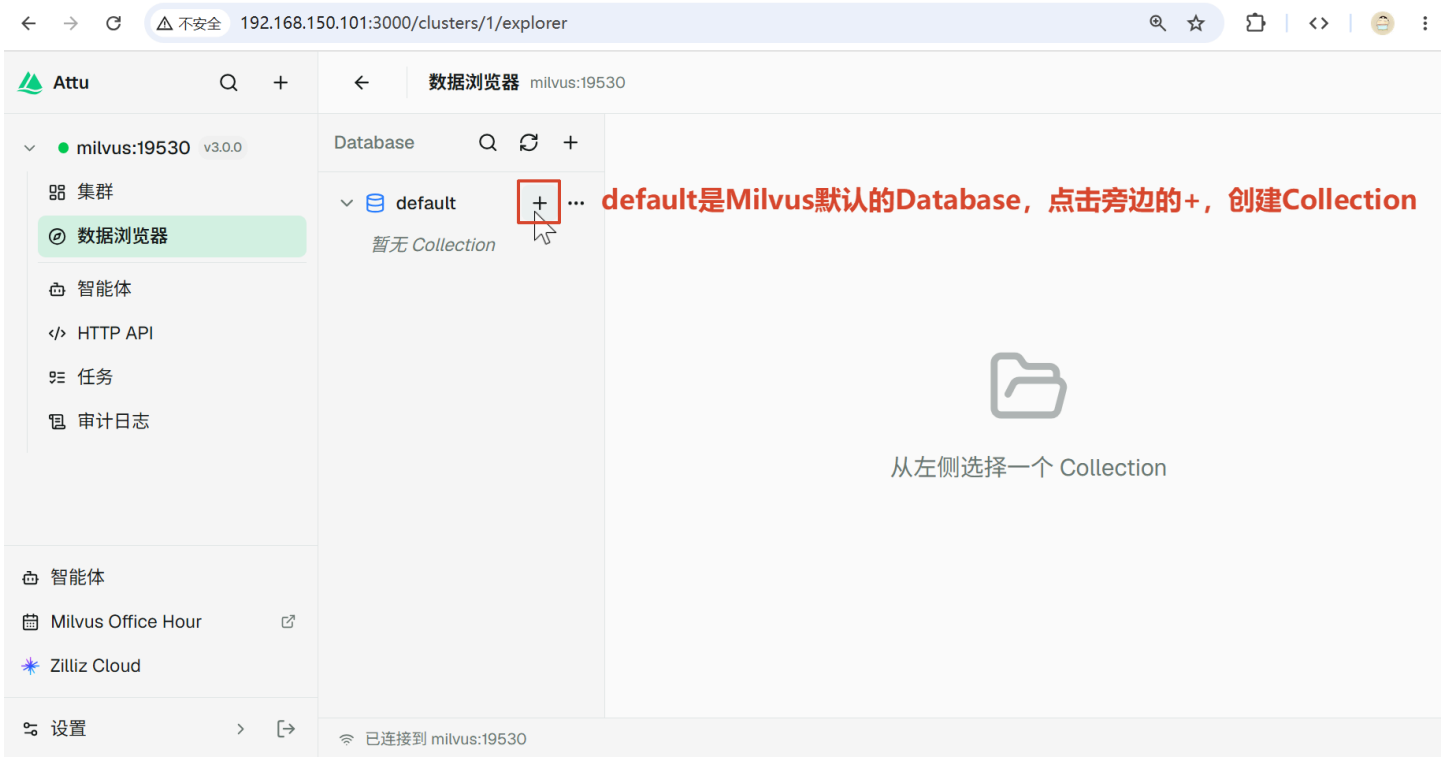
进入后，点击侧边栏的【**数据浏览器**】按钮，可以查看Collection信息：



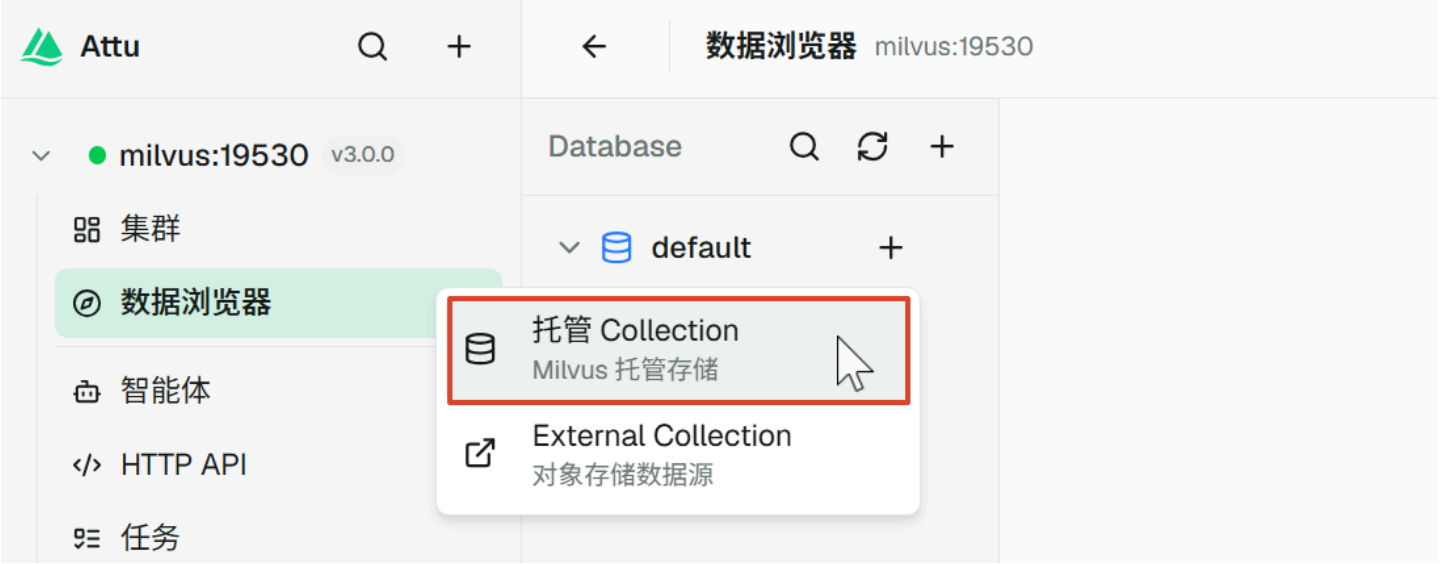
初始状态没有任何Collection

4.2.2 创建Collection

鼠标悬停在【`default`】上，旁边会出现【`+`】，点击即可创建Collection：

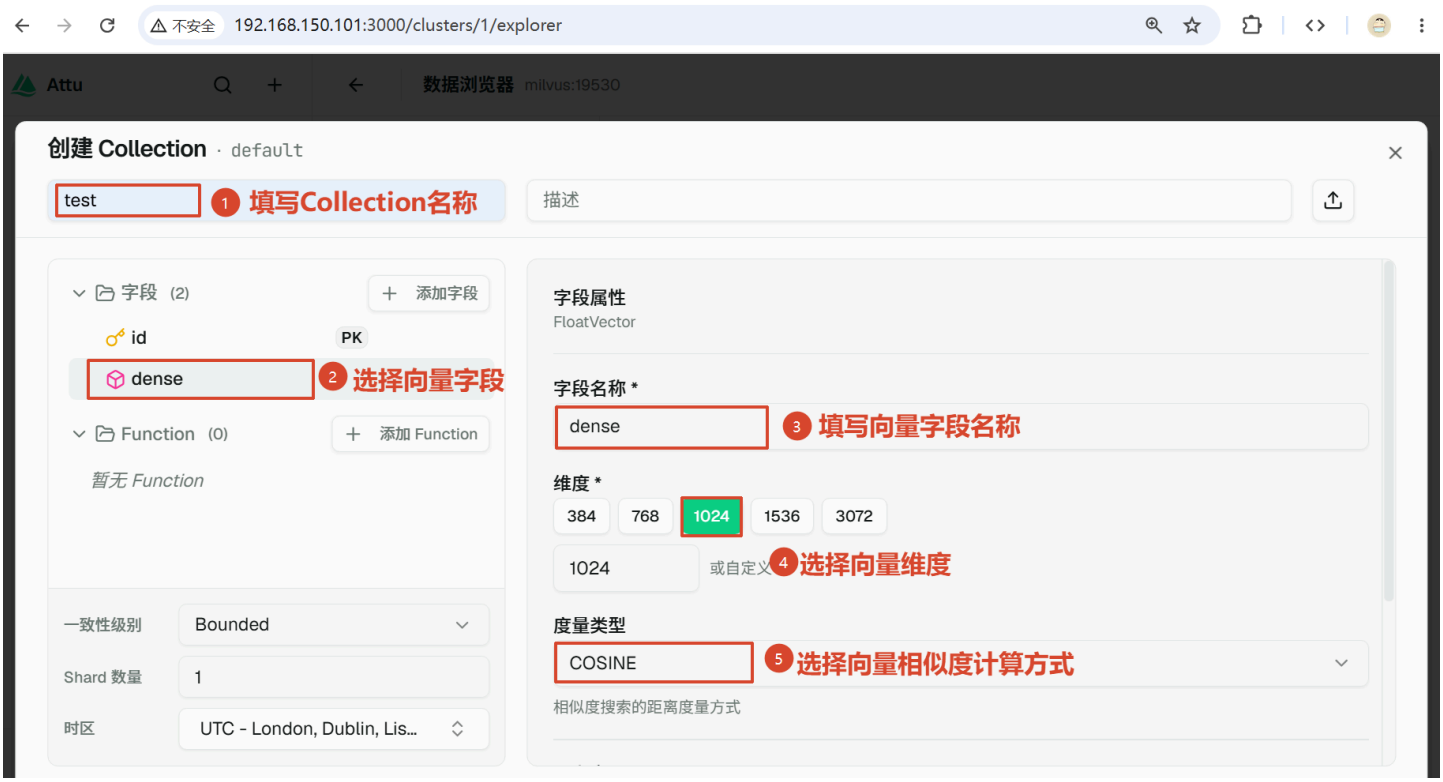


在弹出的窗口中，选择【`托管Collection`】：



A. 填写Collection信息

在弹出的窗口中填写信息：

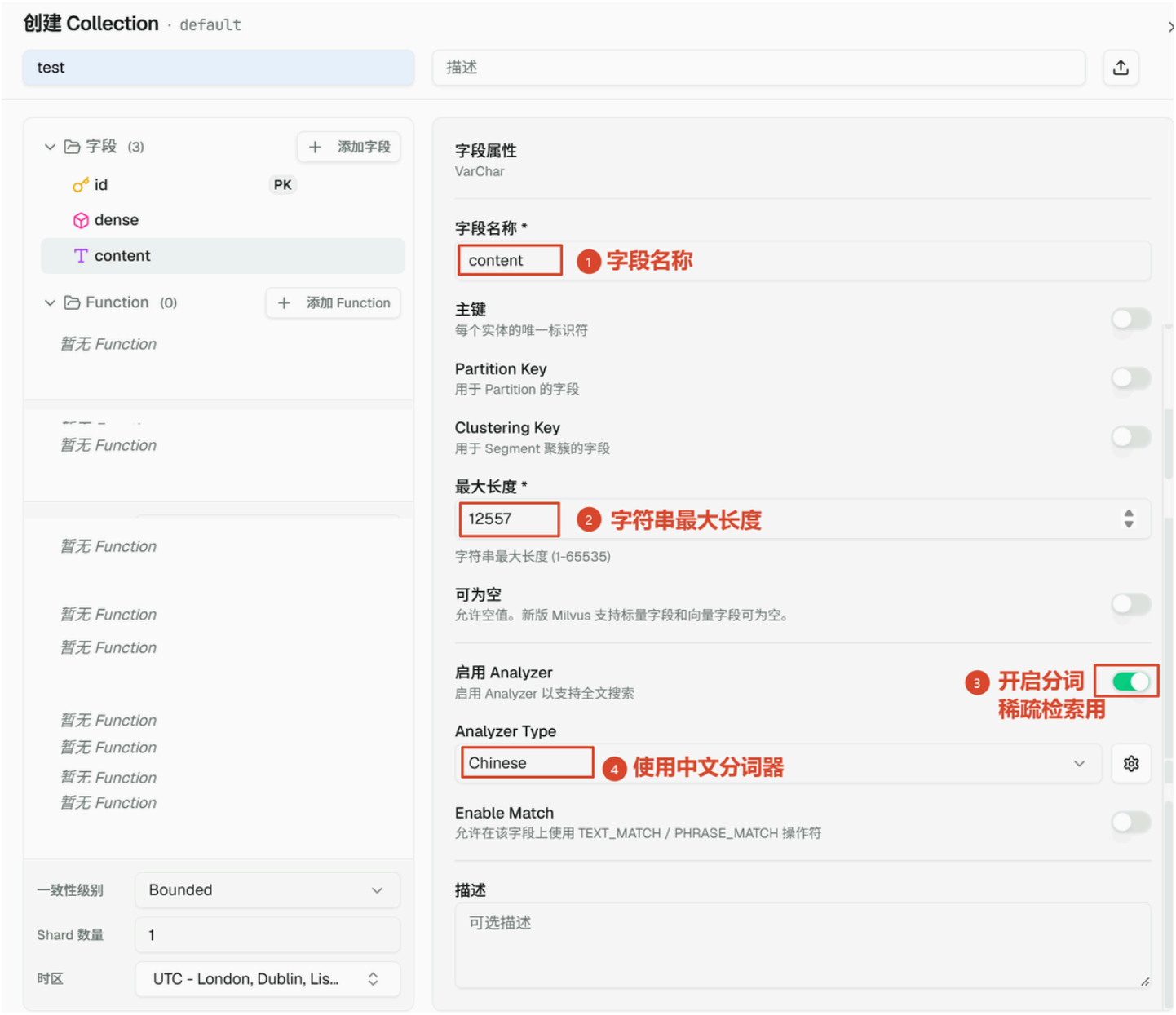


B. 填写字段信息

然后继续添加新字段，例如添加一个存储原始文档内容的字段 `content`，类型选文本-VarChar：


创建 Collection · default
test
描述
字段 (2)
添加字段
id
dense
Function
暂无 Function
主键
向量
数值与布尔
文本与 JSON
数组
其他
Dynamic Field
VarChar
Text
JSON
字段属性
FloatVector
字段名称 *
dense
1024
1536
3072
或自定义
一致性级别
度量类型

继续填写信息：

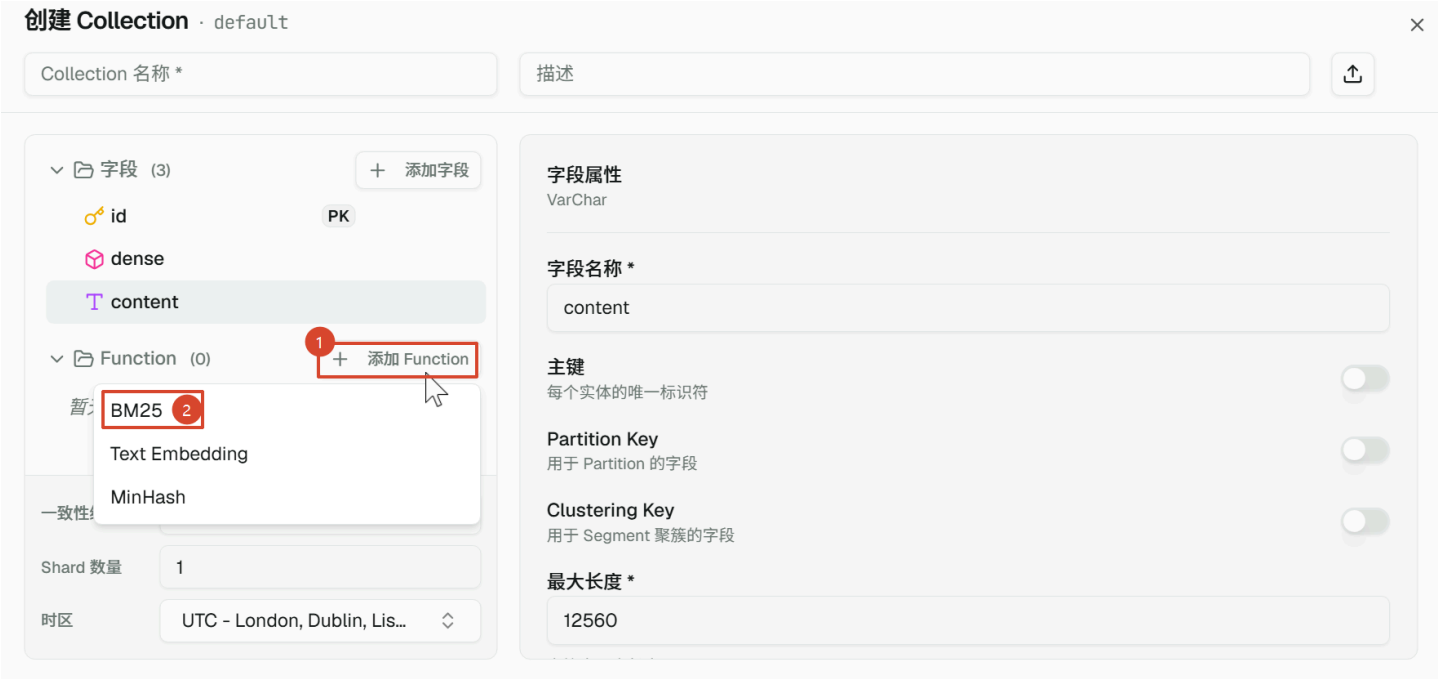

创建 Collection · default
test
描述
字段 (3)
添加字段
id
PK
dense
content
Function (0)
添加 Function
暂无 Function
字段属性
VarChar
字段名称 *
content
1 字段名称
主键
每个实体的唯一标识符
Partition Key
用于 Partition 的字段
Clustering Key
用于 Segment 聚簇的字段
最大长度 *
12557
2 字符串最大长度
字符串最大长度 (1-65535)
可为空
允许空值。新版 Milvus 支持标量字段和向量字段可为空。
启用 Analyzer
启用 Analyzer 以支持全文搜索
3 开启分词
稀疏检索用
Analyzer Type
Chinese
4 使用中文分词器
Enable Match
允许在该字段上使用 TEXT_MATCH / PHRASE_MATCH 操作符
描述
可选描述
一致性级别
Bounded
Shard 数量
1
时区
UTC - London, Dublin, Lis...

C. 填写向量函数

Milvus支持自动的文本转向量，需要定义对应的转换函数：

- BM25_Function：自动把文本转稀疏向量
- Embedding_Function：自动把文本转稠密向量

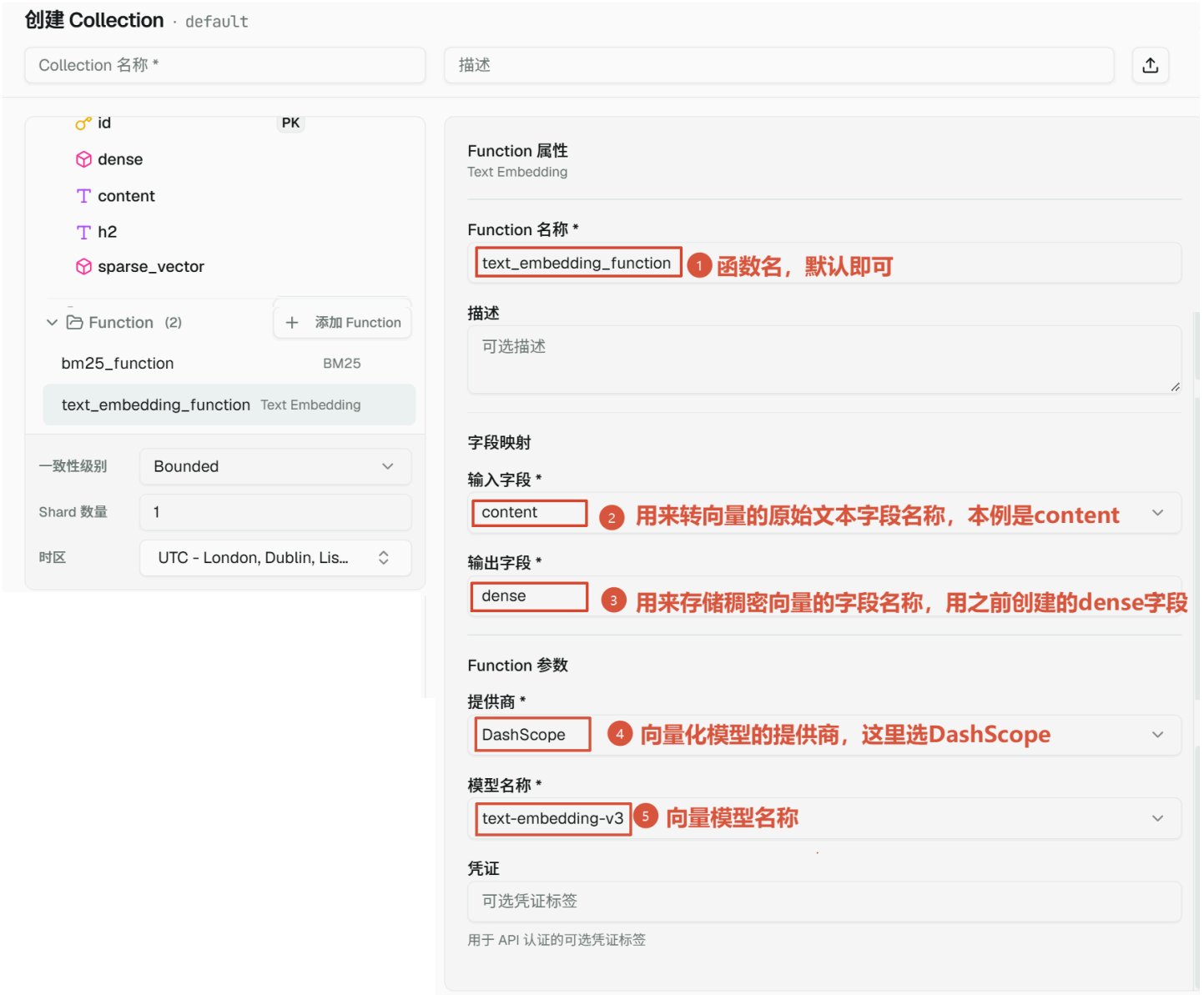
首先是BM25函数：



填写函数信息，最主要是输入字段名称：

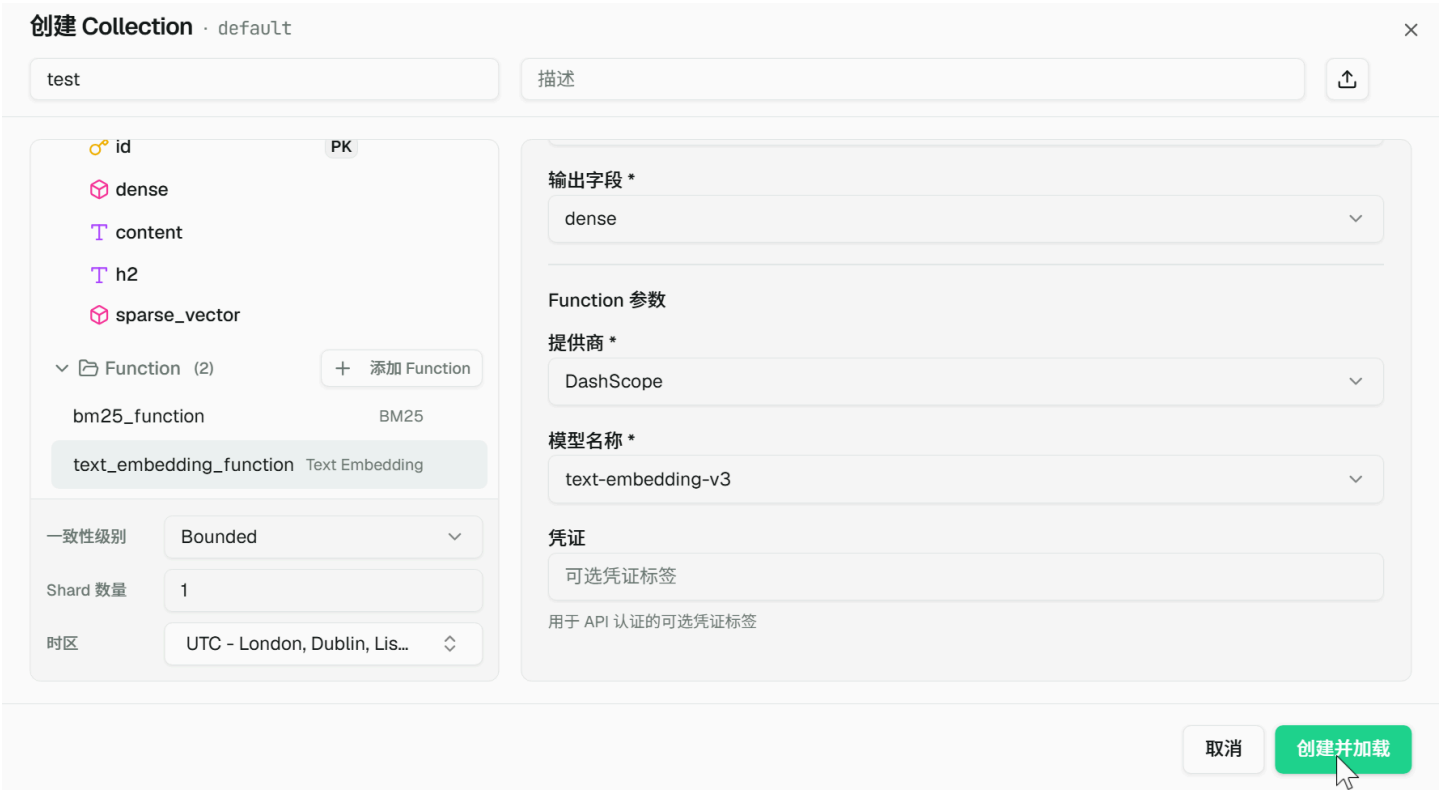


再添加一个 `Embedding_Function`：

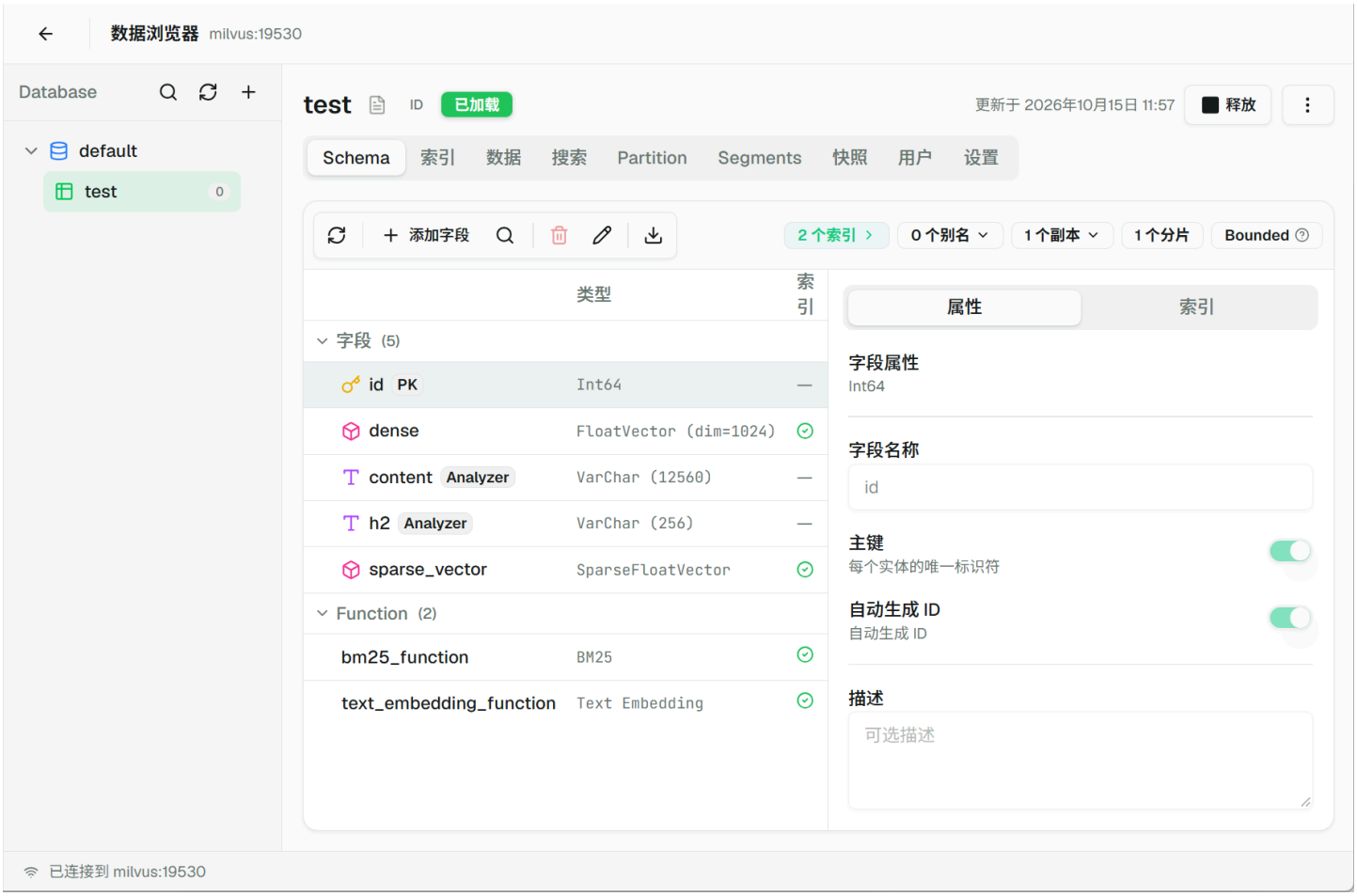


D. 创建Collection

最后，点击创建即可：

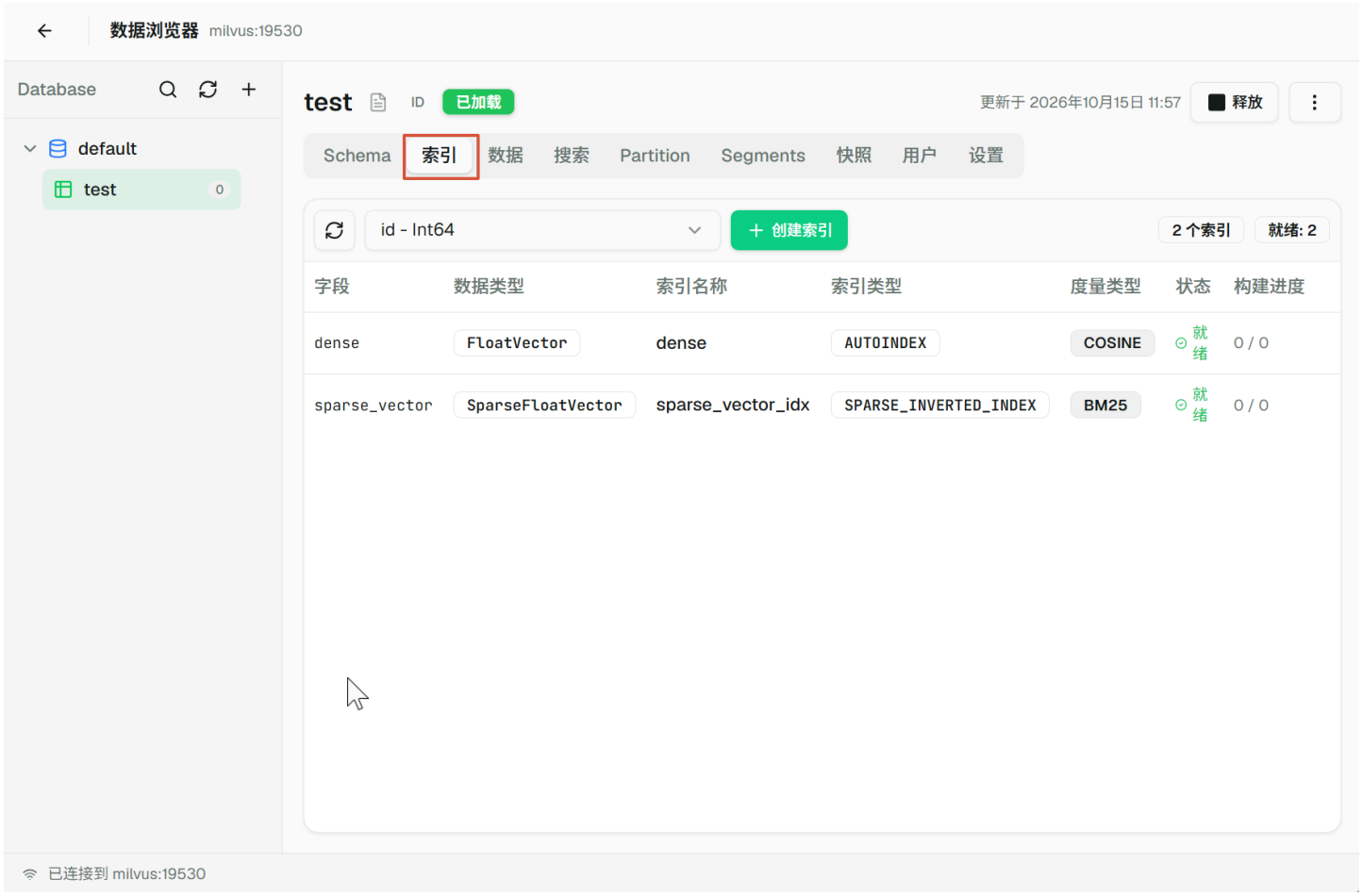


可以看到，Collection创建成功了：



E. 添加索引

点击【**索引**】菜单，可以看到向量字段已经自动添加了索引：



我们可以给任意字段添加新索引：

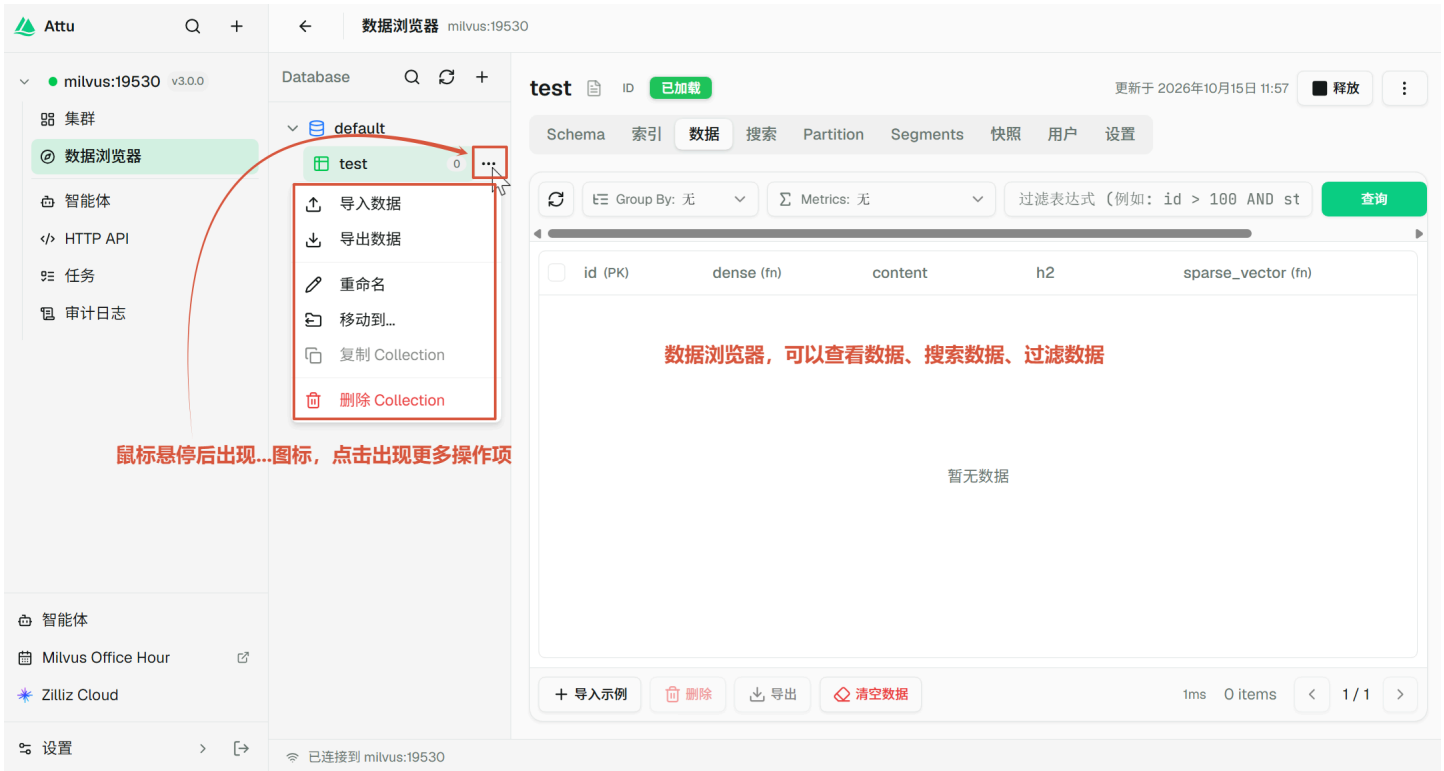


填写索引信息：



4.2.3 其它操作

创建Collection后，就可以向Collection插入数据，或者搜索数据了，当然也可以删除Collection：



4.3 SDK操作Collection

下面我们再看如何用Milvus的SDK操作Collection

4.3.1 连接客户端

新建一个Cell，初始化Milvus的客户端，注意Milvus的地址写成你自己部署的机器IP：

代码块

```
from pymilvus import MilvusClient

# 初始化客户端
client = MilvusClient(uri="http://192.168.150.101:19530")

# Collection名称
collection_name = "my_collection_1"
```

4.3.2 创建collection

示例：

代码块

```
from pymilvus import Function, FunctionType, DataType

def create_collection():
    # 1.schema
    schema = client.create_schema(
        auto_id=False,  # 不自动生成id
        enable_dynamic_field=False,  # 是否允许动态字段
    )

    # 1.1.主键字段
    schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True)
    # 1.2.稠密向量字段
    schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR,
dim=1024)
    # 1.3.稀疏向量字段
    schema.add_field(field_name="sparse",
datatype=DataType.SPARSE_FLOAT_VECTOR)
    # 1.4.普通元数据字段，用来做数据过滤
    schema.add_field(field_name="h2", datatype=DataType.VARCHAR,
max_length=128)
    # 1.5.原始文档内容，需要被向量化
    schema.add_field(
        field_name="content",
        datatype=DataType.VARCHAR,
        max_length=2048,
```

```
        enable_analyzer=True,  # 允许对该字段分词，转BM25稀疏向量
        analyzer_params={"type": "chinese"}  # 指定分词方式，这里选择中文分词
    )
    # 1.6.添加BM25函数，可以自动把原始文档转BM25向量存储
    bm25_function = Function(
        name="text_bm25_emb",  # Function name
        input_field_names=["content"],  # 包含原始文档数据的字段的名称
        output_field_names=["sparse"],  # 稀疏向量字段的名称
        function_type=FunctionType.BM25,  # Set to `BM25`
    )
    schema.add_function(bm25_function)
    # 1.7.添加Embedding函数，可以自动把原始文档转稠密向量存储
    text_embedding_function = Function(
        name="ali",                                          # Embedding函数的名称
        function_type=FunctionType.TEXTEMBEDDING,            # 函数类型，选择 embedding function
        input_field_names=["content"],                       # 用来转稠密向量的原始文本字段的名称
        output_field_names=["dense"],                        # 存储稠密向量字段的名称
        params={                                             # 向量模型提供者的相关参数
            "provider": "dashscope",                         # 模型提供者名称，例如"dashscope"
            "model_name": "text-embedding-v3",               # 向量模型名称
            "dim": "1024",                                   # 其它可选字段：例如向量维度
        }
    )
    # 将配置好的Embedding函数添加到现有的Schema中
    schema.add_function(text_embedding_function)
    # 2.索引
    index_params = client.prepare_index_params()
    # 2.1.稠密向量索引
    index_params.add_index(
        field_name="dense",  # 字段名
        index_type="AUTOINDEX",  # 稠密向量索引方式
        metric_type="COSINE"  # 相似度计算方式，COSINE、IP
    )
    # 2.2.稀疏向量索引
    index_params.add_index(
        field_name="sparse",  # 字段名
        index_type="SPARSE_INVERTED_INDEX",  # 稀疏向量，采用倒排索引
        metric_type="BM25",  # 相似度计算方式，支持 BM25、IP
    )
    # 2.3.普通字段索引
    index_params.add_index(field_name="h2", index_type="AUTOINDEX")

    # 3.创建collection
```

```
    client.create_collection(
        collection_name=collection_name,
        schema=schema,
        index_params=index_params
    )

create_collection()
```

⚡ **说明：**

有关向量字段，我们不是手动把原始文档内容 `content` 转为稀疏向量，而是配置了Milvus内置的 `bm25_function` 和 `embedding_function`，其关键配置是：

- `input_field_names`：输入字段，就是原始文档字段，本例中是 `content`
- `output_field_names`：输出字段，就是保存稀疏或稠密向量的字段，本例中是 `sparse` 和 `dense`

将来，Milvus会在存储阶段和检索阶段自动帮我们生成稀疏向量和稠密向量：

- 存储文档时：
  - `bm25_function` 会自动根据 `content` 内容生成稀疏向量，存入 `sparse` 字段中
  - `embedding_function` 会自动根据 `content` 内容生成稠密向量，存入 `dense` 字段中
- 检索文档时：
  - `bm25_function` 会自动根据用户问题生成稀疏向量，作为检索条件
  - `embedding_function` 会自动根据用户问题生成稠密向量，作为检索条件

但需要特别注意的是，`input_field_names` 指定的字段必须配置两个参数：

- `enable_analyzer=True`：*允许对该字段分词，转BM25稀疏向量*
- `analyzer_params={"type": "chinese"}`：*指定分词方式，这里选择中文分词*

4.3.3 查看Collection

示例：

```python
# 查看Collection
res = client.list_collections()

print(res)  # ['my_collection_1']
```

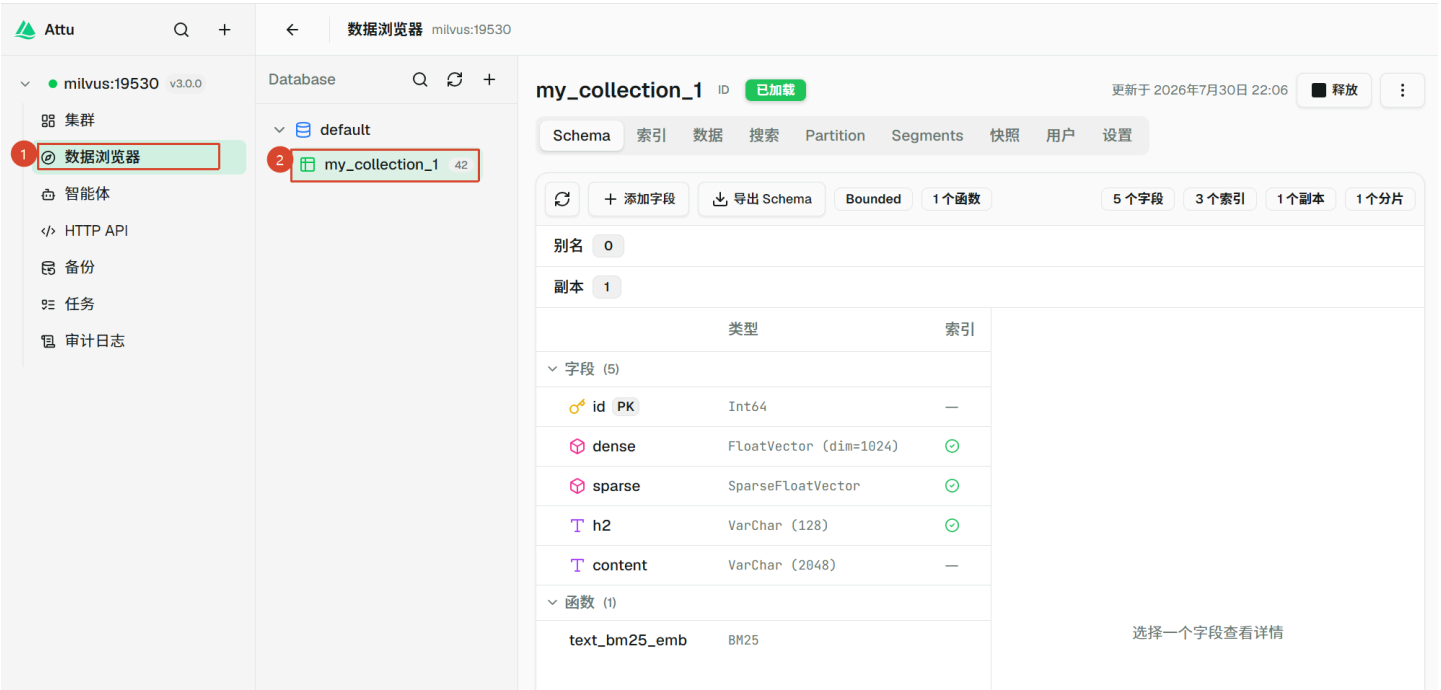
4.4 删除Collection

示例：

代码块

```python
# 删除Collection
client.drop_collection(collection_name=collection_name)
```

进入GUI界面，可以查看新建的Collection



5. Entity操作

准备好了Collection，就可以开始写入文档数据了，也就是Entity.

Entity有三种常见操作：

- `insert`：新增Entity，Entity格式为`dict`，可以是单个，也可以是`list`
- `upsert`：新增或更新，原理是先根据id删除旧的，再填入新的，如果id不存在则等同于新增
- `delete`：删除，可以根据filter条件（非向量字段）删除、根据id删除

5.1 insert

新增数据前先要做数据处理，把LangChain的`Document`处理成Milvus需要的`Dict`格式。

5.1.1 准备数据

我们准备好的文档结构是这样的：

代码块

```
{
  "id": null,
  "metadata": {
    "h1": "第一章 教育概述",
    "h2": "第一节 中外教育家及其教育思想",
    "h3": "（一）教育学萌芽时期代表作"
  },
  "page_content": "昆体良《雄辩术原理》",
  "type": "Document"
}
```

最终我们向Collection中存入的数据结构应该是这样的：

代码块

```
{
    "id": 1,
    "h2": "存h2的内容作为示例",
    "content": "存page_content，也就是文档内容",
    "dense": [0.1, ...], # 存稠密向量
    "sparse": [0.2, ...] # 存稀疏向量
}
```

其中，稠密和稀疏向量可以由创建Collection时定义的`bm25_function`和`embedding_function`来生成，我们无需处理，因此最终写入的数据是这样：

代码块

```
    "id": 1,
    "h2": "存h2的内容作为示例",
    "content": "存page_content，也就是文档内容",
}
```

> ⚡ **注意**
>
> 如果希望用本地模型生成稠密向量和稀疏向量，就需要手动把文档转向量。例如，用ollama_embeddings生成稠密向量：`vectors = embeddings.embed_documents([doc.page_content for doc in docs])`

数据转换示例：

代码块

```
# milvus接收的是dict，所以先做数据转换
data = [
    {
        "id": i+1,
        "content": doc.page_content,
        "h2": doc.metadata.get("h2", ""),
        # "dense": vectors[i],  # 稠密向量字段，如果配置了embedding_function，就无需手动指定
    }
    for i, doc in enumerate(docs)
]
```

5.1.2 插入数据

插入数据调用insert即可：

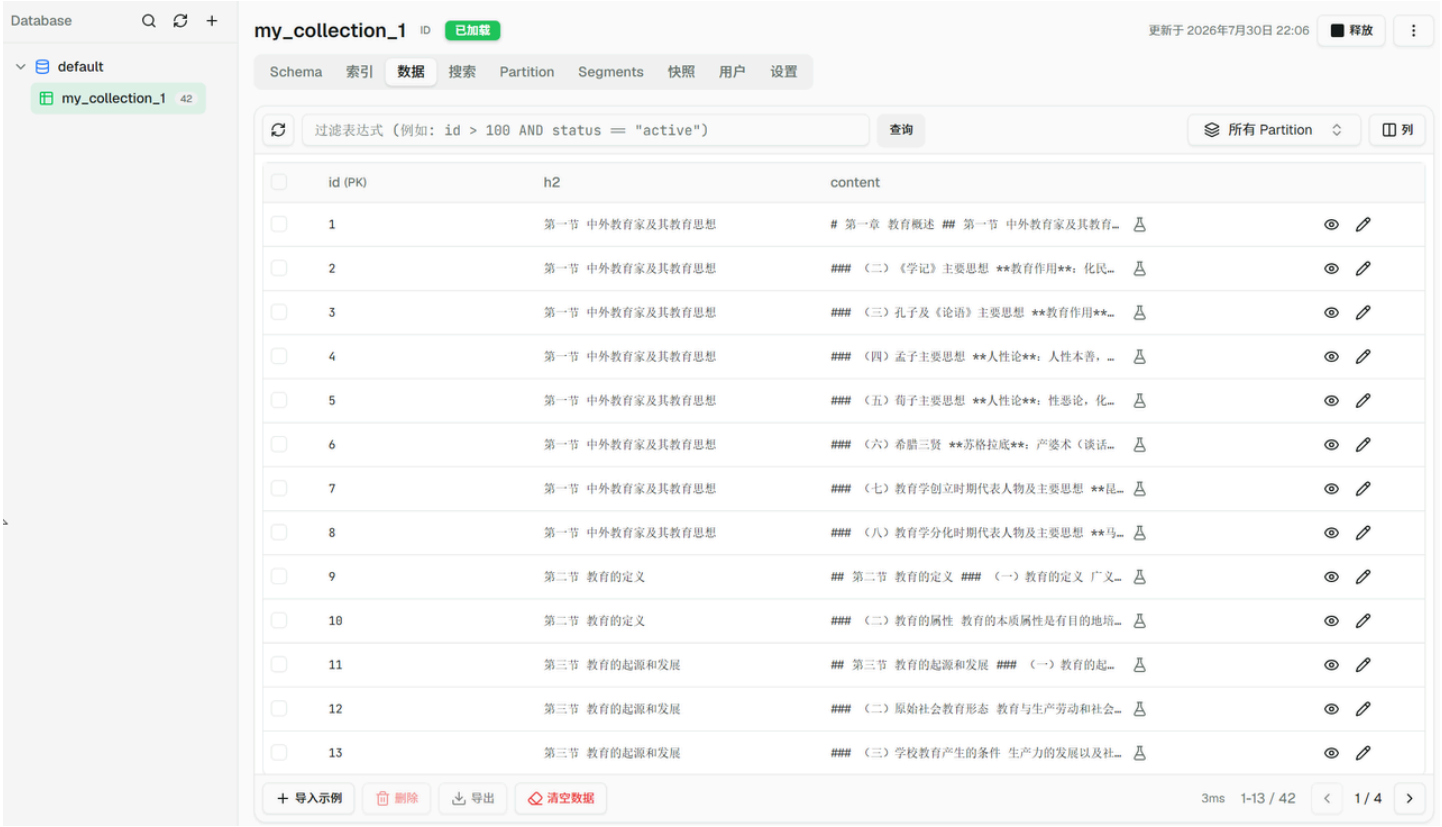
代码块

```
# 插入
res = client.insert(collection_name, data)
print(res)

```

```
#{'insert_count': 21, 'ids': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
15, 16, 17, 18, 19, 20, 21]}
```

查看GUI，也能看到数据已经插入：



5.2 upsert

upsert会先根据id删除旧的，再新增一条数据：

- 如果id存在，等同于更新
- 如果id不存在，等同于新增

代码块

```
# 1.准备两条数据

# id为1的数据，修改其h2字段值，模拟更新
data1 = data[0]
data1['h2'] = "测试一下"

```

```python
# id为2的数据，将其id修改为88，模拟新增
data2 = data[1]
data2['id'] = 88

# 2.测试upsert
res = client.upsert(collection_name, [data1, data2])
print(res)# {'upsert_count': 2, 'ids': [1, 88]}
```

打开GUI控制台，可以看到 `id` 为 `1` 的数据其字段 `h2` 已经被修改：



同时，翻到最后一页，可以看到新增一条id为88的数据：



5.3 delete

代码块

```python
# 根据id删除
res = client.delete(collection_name, ids=[88])
print(res) # {'delete_count': 1, 'cost': 0}
```

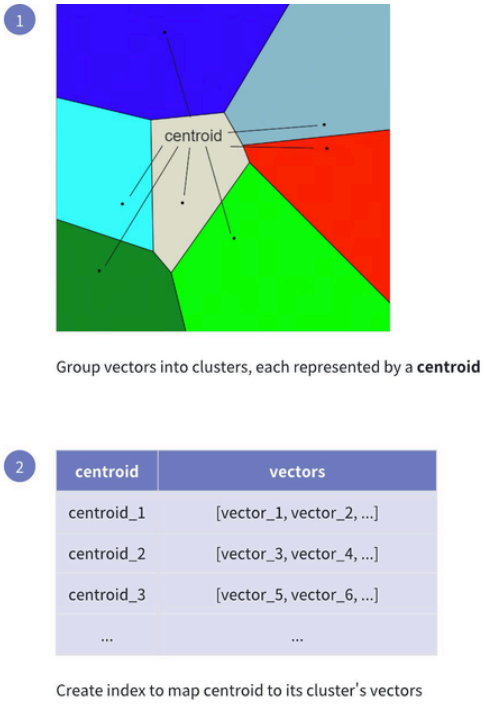
6. Search

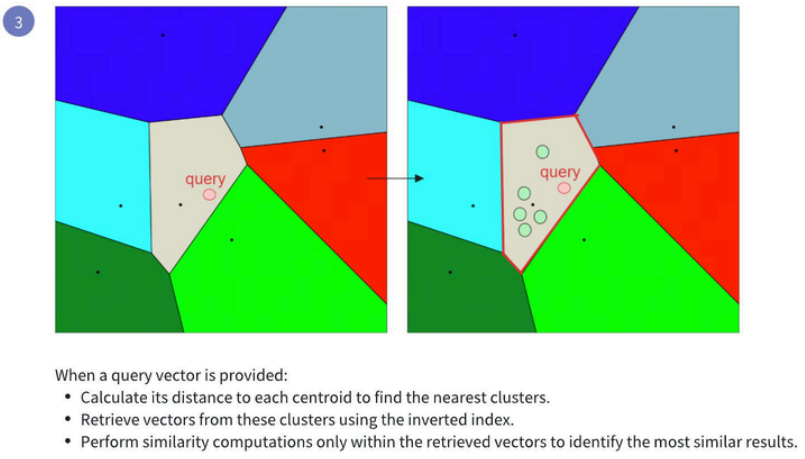
`search` 方法是进行向量相似性搜索的核心接口。Milvus的搜索采用了ANN（近似近邻）算法，搜索效率很高。

ANN 和 k-Nearest Neighbors (kNN) 搜索是向量相似性搜索的常用方法。在 kNN 搜索中，必须将向量空间中的所有向量与搜索请求中携带的查询向量进行比较，然后找出最相似的向量，这既耗时又耗费资源。

与 kNN 搜索不同，ANN 搜索算法要求提供一个索引文件，记录向量 Embeddings 的排序顺序。当收到搜索请求时，可以使用索引文件作为参考，快速找到可能包含与查询向量最相似的向量嵌入的子组。然后，你可以使用指定的度量类型来测量查询向量与子组中的向量之间的相似度，根据与查询向量的相似度对组成员进行排序，并找出前 K 个组成员。

Milvus中ANN的索引方式有很多，比如最常用的：IVF_FLAT算法





6.1 参数速查

其常用参数及核心作用总结如下：

参数名	类型/结构	作用与说明	是否必填
`collection_name`	`string`	指定要执行搜索的目标集合名称。	**是**

`data`	`list` / `list[list]`	查询条件。可以是**单个向量**或包含**多个向量**的批量请求。	**是**
`anns_field`	`string`	指定集合中用于存储向量的字段名称。	**通常必填**
`limit`	`int`	返回与查询向量最相似的结果数量（即 **top-K**）。	**是**
`search_params`	`dict`	配置搜索的详细参数。常用键值对示例： - `"metric_type"`：度量类型，如 `"IP"`（内积）、`"L2"`（欧氏距离）。 - `"offset"`：与 `limit` 配合实现分页，表示跳过的记录数。 - `"radius"`，`"range_filter"`：用于范围搜索。	否
`filter`	`string`	在向量搜索前执行的**标量过滤**表达式（如 `"color == 'red'"`），用于缩小搜索范围。	否
`output_fields`	`list[string]`	指定搜索结果中需要返回的实体字段（如 `["id", "color", "price"]`）。	否
`order_by_fields`	`list[dict]`	允许按标量字段排序（如 `[{"field": "price", "order": "asc"}]`），覆盖默认的相似度排序。	否
`ids`	`list[int]`	通过已存在的主键 ID 替代 `data` 来指定查询向量。**不可与 `data` 同时使用**。	否
`timezone`	`string`	为本次搜索临时设置 IANA 时区（如 `"America/Havana"`），用于处理 `TIMESTAMPTZ` 字段。	否

> **使用要点提醒**：
>
> - 分页时，`limit` 与 `offset` 的总和**不能超过 16384**。
> - `search_params` 中的 `metric_type` 必须与集合索引构建时使用的类型一致。
> - `ids` 与 `data` 是互斥的，只能提供其中一个。

代码块

```
select id, name              # output_fields
from xxx                     # collection_name
where name like "%xx%"       # anns_field data
```

```
4   limit 10
```

6.2 稠密向量检索

我们先定义一个稠密检索的函数：

代码块

```
1   # 定义一个稠密检索的函数
2   def dense_search(query: str):
3       # 1.先把问题向量化
4       query_vector = embeddings.embed_query(query)
5
6       # 2.然后搜索:
7       res = client.search(
8           collection_name=collection_name,     # Collection名称
9           anns_field="dense",                  # 检索字段名（稠密）
10          data=[query_vector],                 # 问题向量（稠密），也可以是问题本身（有向量函数时）
11          limit=3,                             # 查询文档最大数量
12          output_fields=["content", "h2"]   # 要返回的字段名，默认只返回id和score
13      )
14      # 3.遍历结果
15      for hits in res:
16          for hit in hits:
17              print(hit)
```

6.2.1 语义相似查询

稠密检索擅长语义相似度查询，用户输入模糊信息，也能检索到：

代码块

```
1   dense_search("教育的负面作用是什么")
```

结果：

代码块

```
{'id': 15, 'distance': 0.6306518912315369, 'entity': {'content': '# 第二章 教育基本原理 \n## 第一节 教育的功能 \n### （一）个体发展功能和社会发展功能 \n教育的正向功能（积极功能）指教育有助于社会进步和个体发展的积极影响和作用。 \n教育的负向功能（消极功能）指阻碍社会进步和个体发展的消极影响和作用。 \n教育的显性功能是指教育活动依照教育目的，在实际运行中所出现的与之相吻合的结果。 \n教育的隐性功能指伴随显性功能所出现的非预期性的功能。', 'h2': '第一节 教育的功能'}}
{'id': 16, 'distance': 0.5928167104721069, 'entity': {'content': '## 第二节 教育和社会的关系 \n### （一）政治与教育的关系 \n政治（经济制度）对教育具有决定作用，是决定教育性质的直接因素。具体表现为：决定了教育的性质和目的；决定了教育的领导权；决定了哪部分社会成员享有受教育的权利；决定了部分的教育内容；决定了教育的管理体制。 \n教育对政治发展的作用表现在：教育促进人的政治社会化；教育培养现代政治法律人才；教育促进现代政治民主化。', 'h2': '第二节 教育和社会的关系'}}
{'id': 21, 'distance': 0.5773072242736816, 'entity': {'content': '### （六）生产力与教育的关系 \n生产力对其它一切因素都起着决定的作用，是决定教育性质的根本因素。 \n生产力对教育的主要作用表现为：生产力发展水平决定着教育事业发展的规模和速度；生产力发展水平制约着人才培养的规格与教育结构；生产力的发展促进教育内容、教学方法和教学组织形式的发展与改革。 \n教育对生产力的作用表现为：教育再生产劳动力；教育是科学知识与技术发展的重要手段。', 'h2': '第二节 教育和社会的关系'}}
```

6.2.2 专有名词查询

如果用户搜索的内容是专有名称，就不一定能准确检索了：

代码块

```
# 测试，《给教师的建议》是书的名字，属于专业名词
dense_search("给教师的建议是谁写的")
```

结果：

代码块

```
{'id': 1, 'distance': 0.48752158880233765, 'entity': {'content': '# 第一章 教育概述 \n## 第一节 中外教育家及其教育思想 \n### （一）教育学萌芽时期代表作 \n国内：《学记》，世界最早，成文于战国末期，作者是乐正克。 \n国外：昆体良《雄辩术原理》/《论演说家的教育》。', 'h2': '第一节 中外教育家及其教育思想'}}
{'id': 6, 'distance': 0.45305097103118896, 'entity': {'content': '### （六）希腊三贤 \n**苏格拉底**：产婆术（谈话法），分三步：讽刺—定义—助产术。国外启发式教育第一人。 \n**柏拉图**：《理想国》提出普及教育的主张，教育目的是培养哲学王。 \n**亚里士多德**：提出自由教育，提倡对学生进行和谐全面发展的教育，灵魂说。', 'h2': '第一节 中外教育家及其教育思想'}}
```

6.3 稀疏向量检索

稀疏检索刚好与稠密检索相反，擅长关键词匹配，而不擅长语义分析。

我们先定义一个稀疏检索函数：

代码块

```
# 定义稀疏检索函数
def sparse_search(query: str):
    # 1.直接检索，内置的hm25函数会自动把query转向量
    res = client.search(
        collection_name=collection_name,  # Collection名称
        data=[query],
        anns_field="sparse",  # 检索字段名（稀疏）
        limit=3,  # 查询文档最大数量
        output_fields=["content", "h2"]  # 要返回的字段名
    )
    # 2.打印结果
    for hits in res:
        for hit in hits:
            print(hit)
```

6.3.1 语义相似查询

如果用户搜索内容与文档意思接近，但内容不同，往往检索不到：

代码块

```
# 测试模糊的语义搜索，稀疏检索不太擅长
sparse_search("教育的负面作用是什么")
```

结果：

代码块

```
{'id': 21, 'distance': 1.9043395519256592, 'entity': {'content': '###  （六）生产力与教育的关系  \n生产力对其它一切因素都起着决定的作用，是决定教育性质的根本因素。  \n生产力对教育的主要作用表现为：生产力发展水平决定着教育事业发展的规模和速度；生产力发展水平制约着人才培养的规格与教育结构；生产力的发展促进教育内容、教学方法和教学组织形式的发展与改革。  \n教育对生产力的作用表现为：教育再生产劳动力；教育是科学知识与技术发展的重要手段。', 'h2': '第二节  教育和社会的关系'}}
{'id': 20, 'distance': 1.871191143989563, 'entity': {'content': '###  （五）人口与教育的关系  \n人口对教育的作用主要表现在：人口的数量和增长是决定教育事业的规模、速度的一个重要因素；人口的质量影响教育的质量；人口的结构影响着教育的发展；人口流动影响着教育的发
```

```
展。  \n教育对人口的作用主要表现在：控制人口的数量；提高人口的质量；改善人口结构；调整人才构成与流动等作用。', 'h2': '第二节 教育和社会的关系'}}
3  {'id': 5, 'distance': 1.8368871212005615, 'entity': {'content': '### （五）荀子主要思想  \n**人性论**：性恶论，化性起伪，善德是后天习得的，重视教育的作用。  \n**教学原则**：学以致用，锲而不舍。', 'h2': '第一节 中外教育家及其教育思想'}}
```

6.3.2 专有名词查询

如果用户搜索的内容是专有名称，恰好是稀疏检索擅长的：

代码块

```
1  # 专有名称查询:
2  sparse_search("给教师的建议是谁写的")
```

结果：

代码块

```
1  {'id': 8, 'distance': 5.089690685272217, 'entity': {'h2': '第一节 中外教育家及其教育思想', 'content': '### （八）教育学分化时期代表人物及主要思想  \n**马卡连柯**：著有《教育诗》，提出集体主义教育。  \n**克鲁普斯卡娅**：著有《国民教育与民主制度》，是最早以马克思主义为基础探讨教育问题的教育家。  \n**杨贤江**：著有《新教育大纲》，是中国第一部以马克思主义为指导的教育学著作。  \n**凯洛夫**：著有《教育学》，是世界第一部马克思主义的教育学著作。  \n**布鲁姆**：著有《教学目标分类学》，提出了掌握学习理论：所有学生都能学好；目标分为认知、情感、动作技能。  \n**布鲁纳**：著有《教学过程》，提出了结构主义教学理论，倡导发现式学习。  \n**瓦·根舍因**：著有《范例教学理论》，与布鲁纳和赞可夫被认为课程现代化的三大代表人物。  \n**赞可夫**：著有《教学与发展》，提出了发展性教学理论的五原则：高难度、高速度、理论知识起主导作用、理解学习过程、所有学生包括差生都得到发展的原则。  \n**苏霍姆林斯基**：著有《给教师的建议》《把整个心灵献给孩子》《帕夫雷什中学》，提出个性全面和谐发展的教育思想，他的著作被称为“活的教育学”。  \n**巴班斯基**：著有《教学过程最优化》《教学教育过程最优化》，把现代控制论、系统论观点用于教学论研究，提出教学过程最优化的理论。  \n**皮亚杰**：提出儿童认知发展阶段论、道德发展阶段论，以及儿童与环境互相作用的两个过程：同化、顺应。  \n**维果斯基**：提出心理学的“文化–历史”发展理论；最近发展区理论，教学必须走在发展的前面；支架式教学；“内化”学说。'}}
2  {'id': 7, 'distance': 2.6363399028778076, 'entity': {'h2': '第一节 中外教育家及其教育思想', 'content': '### （七）教育学创立时期代表人物及主要思想  \n**昆体良**：西方第一个专门论述教育问题的教育家。  \n**培根**：在《论科学的价值和发展》中首次把“教育学”作为一门独立的科学确立下来。  \n**夸美纽斯**（教育学之父、教育史上哥白尼）：代表作《大教学论》是教育学成为一门独立学科的标志。提出“泛智教育”、“班级授课制”和“学年制”；把教师比喻为太阳底下最光辉的事业。  \n**康德**：教育学作为一门课程在大学里讲授，始于康德。  \n**赫尔巴特**（传统教育代表、科学教育学之父）：《普通教育学》是教育学作为一门规范、独立的学科正式诞生的标志。提出以伦理学和心理学作为教育学的基础；教学要有教育性；三中心论：教师、教材、课堂；四阶段论：清楚、联想、系统和方法。  \n**杜威**（现代教育代表、实用主义哲学之父、儿童
```

```
中心主义论、教育无目的论）：《民主主义与教育》提出教育的本质：教育即生活、教育即生长、教育即经验的改组或改造；学校即社会；从做中学；连带学习；三中心论：儿童、经验、活动；五步教学法：设疑–分析–假设–推断–验证。 \n**卢梭**：《爱弥儿》，提倡自然主义教育思想，认为教育的任务应该使儿童归于自然。 \n**洛克**：《教育漫画》，提出白板说，倡导绅士教育。 \n**裴斯泰洛奇**：《林哈德与葛笃德》，教育心理学化。 \n**斯宾塞**：反对思辨，主张用实证方法研究知识价值；生活预备说；科学知识最有价值，制定以科学知识为核心的课程体系。 \n**陶行知**：提出生活教育理论：生活即教育，社会即学校，教学做合一；“捧着一颗心来”（师德）；伟大的人民教育家。 \n**蔡元培**：思想自由，兼容并包；以美育代宗教；学界泰斗，人世楷模。'}}
3 {'id': 3, 'distance': 2.613140344619751, 'entity': {'h2': '第一节 中外教育家及其教育思想', 'content': '### （三）孔子及《论语》主要思想 \n**教育作用**：庶、富、教；性相近，习相远。 \n**教育对象**：“有教无类”，教育民主思想。 \n**教育目的**：以完善人格为教育的首要目的，培养士和君子。 \n**教学内容**：文、行、忠、义。 \n**教学过程**：学、思、习、行。 \n**教学原则**：因材施教、启发诱导、学思结合、谦虚笃实。 \n**教师观**：其身正，不令而行；其身不正，虽令不从。'}}
```

6.4 混合检索

正如前面所演示的，单一采用稀疏或稠密检索都会存在问题:

- **稀疏检索**: 可以精确匹配专业名词，但会漏掉同义词或语义接近的内容
- **稠密检索**: 擅长检索语义接近的内容，但可能错过专业名词，甚至错过关键词

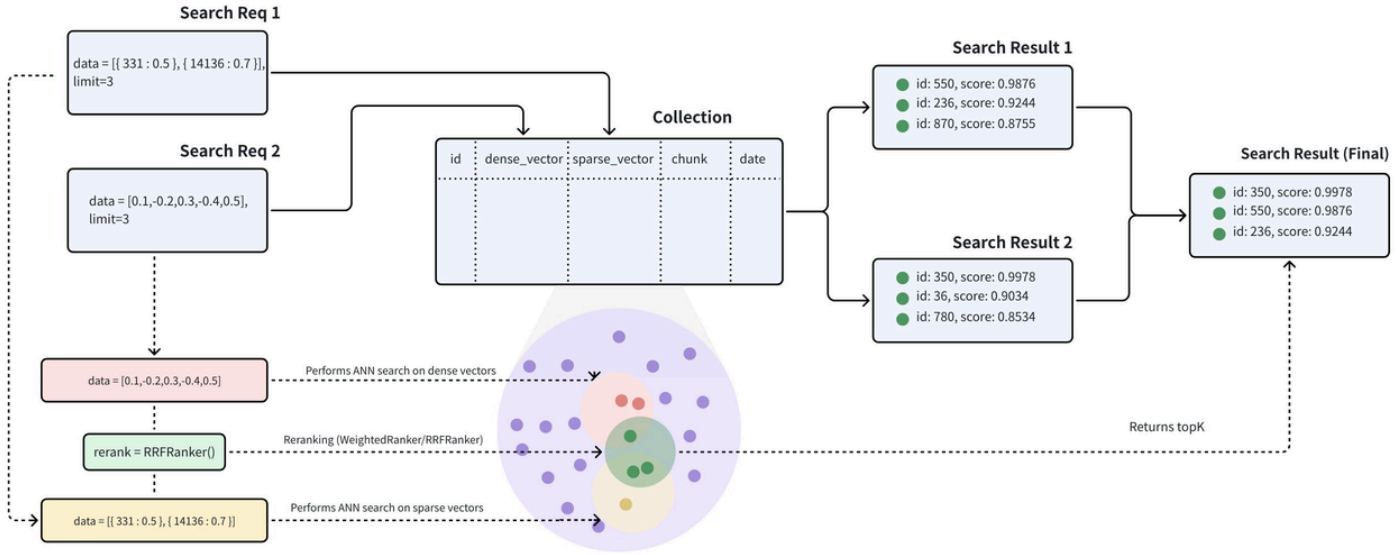
所以，最佳的方案就是把两种方案结合，也就是**混合检索**。

6.4.1 混合检索流程

混合检索流程是这样的：

1. **多路召回**：通过稀疏、稠密等多种方式分别查询文档列表，得到多个结果集
2. **融合重排**：再将所有文档融合为一个列表，重新打分排序

混合搜索结合了稠密、稀疏等方法，既能提供广泛的语义理解，又能提供精确的术语查询。混合搜索充分利用了每种方法的优势，克服了单独方法的局限性，为复杂查询提供了更好的性能。



但问题来了：

> 多路召回的文档结果，该如何融合重排呢？

常见的融合重排方式有：

- **加权求和:** 首先需要对分数归一化，然后给不同的召回方案设置不同权重，然后合并分数
- **RRF**：基于倒数排名求和，只与排名有关，与分数无关。公式：1/(k + rank)
- **Cross-Encoder**：利用神经网络模型，对多路结果做精排

6.4.2 加权融合

加权融合顾名思义，就是给多路召回的文档分配不同的权重，然后将文档在每一路中的打分加权求和，重新排序。

6.4.2.1 原理

例如，我们设定**稠密**与**稀疏**的权重比例为：`[0.6, 0.4]`，假设一篇文档得分如下：

- 稠密检索得分：0.5
- 稀疏检索得分：0.8

最终得分就是：

```
代码块
```

```
0.5 * 0.6 + 0.8 * 0.4 = 0.62
 ┬   ┬   ┬   ┬
 │   │   │   │
 │   │   │   └──稀疏权重
 │   │   └──稀疏得分
 │   │
 │   └──稠密权重
 └── 稠密得分
```

不过，需要注意的是，由于不同的检索体系打分方式不同，分值差异可能较大：

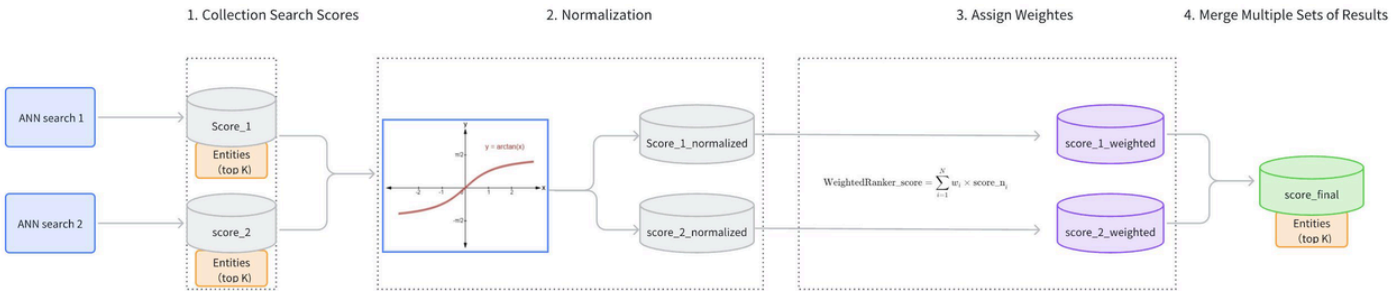
检索器	分数范围	典型值
BM25	通常 0 ~ 10+	0.5, 1.2, 5.0
向量余弦相似度	0 ~ 1（或 -1~1）	0.85, 0.92

直接做加权求和没有意义，分数尺度大的检索器会主导结果。

通常，我们都会把**分数归一化**，使所有分数分布到特定范围，比如 `[0,1]`，常见的归一化方法有：

- **Min-Max**: 计算简单，值范围固定为 `[0,1]`，为公式为 score - min / (max - min)
- **Z-Score**: 转为均值为0，标准差为1的正态分布。公式是 (s - avg) / 标准差
- **Softmax**: 将分数转为概率分布，所有值在 0~1 之间，且总和为 1，分布均匀程度取决于 temperature

整体流程如图：



6.4.2.2 实现

Milvus默认支持混合检索，基本步骤如下：

- 创建多路请求，比如：稀疏、稠密
- 配置融合策略，比如：加权、RRF
- 发起请求

我们先定义一个混合检索函数，但融合重排策略作为参数传递：

代码块

```
# RRF 混合检索
from pymilvus import AnnSearchRequest

def hybrid_search(query: str, ranker):
    # 1.创建稠密、稀疏请求
    # 稠密
    dense_request = AnnSearchRequest(
        data=[query],
        anns_field="dense",
        param={},
        limit=3,
    )
    # 稀疏
    sparse_request = AnnSearchRequest(
        data=[query],
        anns_field="sparse",
        param={},
        limit=3,
    )
    # 混合在一起
    reqs = [dense_request, sparse_request]

    # 2.设定融合策略，发送请求
    res = client.hybrid_search(
        collection_name=collection_name,
        reqs=reqs,
        ranker=ranker,
        limit=2,
        output_fields=['content', 'h2']
    )
    for hits in res:
        print("TopK results:")
        for hit in hits:
            print(hit)
```

然后使用Milvus提供好的权重融合策略：

代码块

```
from pymilvus import WeightedRanker

# 定义加权融合策略
ranker = WeightedRanker(0.5, 0.5)
# 测试
hybrid_search("教育的负面作用是什么", ranker)
```

6.4.3 RRF

RRF（Reciprocal Rank Fusion，倒数排名融合） 是一种用于融合多个检索结果列表的算法，它与多路召回的文档得分无关，只关心排名。

	稠密	稀疏
A	1	6
B	4	3

核心公式： $$RRF(d) = \sum_{r=1}^{n} \frac{1}{k + \text{rank}_r(d)}$$

符号	含义
d	某个文档
k	常量，通常是60
$\text{rank}_r(d)$	文档 d 在第 r 个检索结果列表中的排名位置（从1开始）
$\text{RRF}(d)$	文档 d 的最终融合得分

RRF 的特点：

优点	缺点
无需归一化，因为与分数无关	对排名靠后的文档不敏感
对离群值不敏感	无法处理得分/置信度信息
计算简单、效率高	
可融合任意数量的检索列表	所有检索源权重相同

Milvus同样提供了默认的RRF融合策略，用法如下：

代码块

```
from pymilvus import AnnSearchRequest, RRFRanker

# 定义RRF融合策略
ranker = RRFRanker(k=60)
# 测试
hybrid_search("教育的负面作用是什么", ranker)
```

6.4.4 Cross-Encoder

Cross-Encoder，交叉编码，把用户问题和每个文档一起输入模型，精确打分。

与之相对的是：**Bi-Encoder**, 并行编码，把用户问题、文档分别交给模型，转为向量，然后比较相似度。

Bi-Encoder由于是把文档和问题分开处理，所以可以把文档向量化放在离线阶段，存入向量库。检索时只需要把问题向量化，然后与文档向量比较即可。所以速度更快，能处理数以百万的文档，但精度略差，适合**初筛**。

Cross-Encoder需要实时把文档和问题一起计算，效率较低，但精度较高，适合**精排**。

我们之前所说的稀疏向量检索、稠密向量检索都属于是Bi-Encoder，相当于对文档做初步筛选。筛选后的结果再交给专业的Cross-Encoder模型做精排，检索的结果会更加精准。

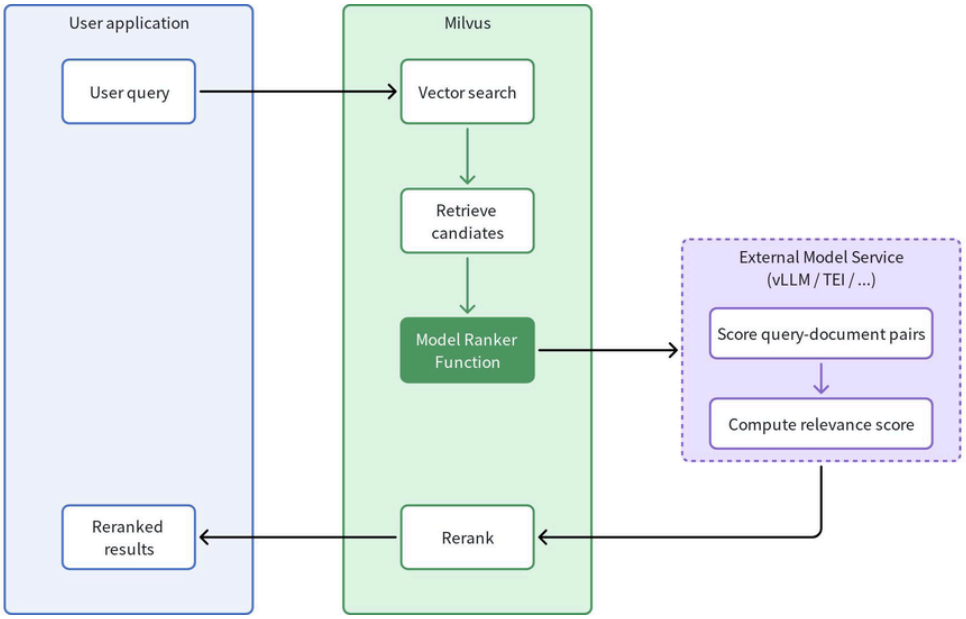
Cross-Encoder需要用专业的**Reranker**模型，比如：

模型名称	研发机构	发布时间	参数量	上下文长度	授权协议	核心亮点
Qwen3-Reranker-4B	阿里通义	2025-06	4B	32k	Apache 2.0	MMTEB-R 评分 72.74（高精度），支持32K长文本，中英文均衡适配，适合企业级高精度RAG
mxbai-rerank-large-v2	MixedBread	2025-07	1.5B	8k	Apache 2.0	BEIR 18项任务零样本SOTA，完美适配德/英/西/法等多语言场景，泛化性强
bge-reranker-v2-m3	BAAI	2024-03	567M	8k	Apache 2.0	中文社区主流选择，量化后体积<200MB，中英文混合场景表现突出，部署成本低
jina-reranker-v3	Jina AI	202510	0.6B	默认8k	CC-BY-NC-4.0（商用需授权）	支持100+语言，专为Agentic-RAG场景微调，listwise交叉交互机制，适配多轮检索决策
ms-marco-MiniLM-L6-v2	Microsoft	2021	22M	512	MIT	超轻量架构，CPU/边缘端部署首选，英文通用场景标杆模型，推理速度快、集成成本低
ColBERT v2	CMU/Stanford	2021-12	110M	原生512	MIT	预编码+后期交互架构，百万级文档低延迟重排，吞吐较传统模型提升10倍，适合大规模英文知识库，长文本需分块处理

6.4.4.1 实现方案

Milvus天然支持Cross-Encoder的重排序方案，不过由于Cross-Encoder方案依赖于Rerank模型，所以需要你有可访问的Rerank模型服务才行。

Milvus的Cross-Encoder重排实现流程是这样的：



流程说明：

1. 初始查询：您的应用向 Milvus 发送查询请求
2. 向量检索：Milvus 执行标准向量检索（可以是混合检索模式），以识别候选文档
3. 候选召回：系统根据向量相似度，初步筛选出一组候选文档
4. 模型评估：Rerank模型（Model Ranker Function）对（**query-document**）**对**进行处理：
   - 将Query和Document发送至外部模型服务
   - Rerank模型评估Query与每个Document之间的语义相关性
   - 每个Document基于语义理解获得一个相关性分数
5. 智能重排：根据模型生成的相关性分数对文档重新排序
6. 结果增强：您的应用收到的结果是按语义相关性排序的，而非仅依赖向量相似度

Milvus支持的外部Reranker模型有很多种，可以是自己部署的，也可以是云服务的：

提供商	最适合	特点
vLLM	需要深度语义理解和定制化的复杂应用	• 支持各种大型语言模型 • 灵活的部署选项 • 更高的计算要求 • 更大的定制潜力
TEI	快速实施，资源利用率高	• 针对文本操作进行优化的轻量级服务 • 部署更简便，资源需求更低

		• 预优化重新排序模型 • 基础设施开销极低
Cohere	优先考虑可靠性和集成便捷性的企业级应用	• 企业级可靠性和可扩展性 • 无需基础设施维护的托管服务 • 多语言重新排序功能 • 内置速率限制和错误处理
Voyage AI	具有特定性能和上下文要求的RAG应用	• 专门针对重新排序任务训练的模型 • 针对不同文档长度的精细截断控制 • 针对生产环境工作负载优化的推理 • 多种模型变体（rerank-2、rerank-lite等）
SiliconFlow	优先考虑成本效益的長文档处理应用	• 支持可配置重叠度的先进文档分块技术 • 基于片段的评分 • 支持多种重新排序模型 • 提供标准版和专业版两种模型，性价比高
DashScope	使用阿里云或Qwen重新排序模型的应用程序 Milvus 3.x版本才支持	• 托管式 DashScope 重新排序 API • 支持以下重新排序模型：`gte-rerank-v2` • 基于 API 密钥的身份验证
Hugging Face	使用托管版 Hugging Face 句子相似度模型的应用程序	• 使用托管版 `hf-inference` 提供商 • 从 Hugging Face Hub 中选择模型 • 为每个候选项计算一个句子相似度分数 • 使用 API 密钥进行身份验证

有关各模型服务实现的详细信息，请参阅相关文档：

- vLLM Ranker
- TEI Ranker
- Cohere Ranker
- Voyage AI Ranker
- SiliconFlow 排名器
- DashScope 排名系统

- Hugging Face 排名器

> ⚡ **注意：**
>
> DashScope的重排方案仅在Milvus3.0.x的版本才开始才支持，如果是小于这个版本，建议在企业中使用vLLM部署私有模型。

下面，我们以DashScope为例来说明这种重排方案的用法。

6.4.4.2 配置密钥

不管使用哪种模型提供商，都需要在Milvus服务中配置授权密钥，也就是API_KEY.

找到虚拟机中的 `deploy/milvus/user.yaml` 文件，将其中的 `credential.dashscope_apikey.apikey` 改成你自己的DashScope的 `API_KEY`：

代码块

```
credential:
  dashscope_apikey:
    apikey: sk-xxxxxxx
```

然后重新启动milvus：

代码块

```
cd /root/deploy

docker compose restart milvus
```

6.4.4.3 定义RerankFunction

接着，我们回到Notebook，定义一个Cross-Encoder的Function：

代码块

```
# cross-encoder rerank
from pymilvus import Function, FunctionType
```

```
3
4
5   def create_cross_encoder_ranker(query: str):
6       return Function(
7           name="dashscope_semantic_ranker",       # ranker名词，唯一即可
8           input_field_names=["content"],           # 原始文档字段
9           function_type=FunctionType.RERANK,  # ranker类型，这里是固定值
10          params={
11              "reranker": "model",  # rerank类型，基于模型rerank，也就是cross-
    encoder
12              "provider": "ali",  # rerank模型提供者
13              "model_name": "gte-rerank-v2",  # rerank模型名称
14              "queries": [query],  # 查询条件
15              "max_client_batch_size": 5,  # 单批次中处理的文档最大数量
16          },
17      )
```

这段代码是通用的代码，将来你的模型提供商会变化，但这段代码基本不变。

⚡ Milvus中Cross-Encoder这种策略的Function定义是固定代码，变化的仅仅是模型的提供商、模型的名称。核心代码参数是这样的：

- `name`：*ranker名词，自定义，但要唯一*
- `input_field_names`：*用来存放原始文档的字段*
- `params`：*ranker的核心参数*
  - `reranker`：*rerank类型，支持model、rrf、weight等，model就是cross-encoder模式*
  - `provider`：*模型的提供商，就是上面说的几个*
  - `model_name`：*rerank模型名称*
  - `queries`：*查询条件*
  - `max_client_batch_size`：*单批次中处理的文档最大数量*

6.4.4.4 基于Cross-Encoder的混合检索

最后，沿用之前的混合检索函数，传递`create_cross_encoder_ranker`即可：

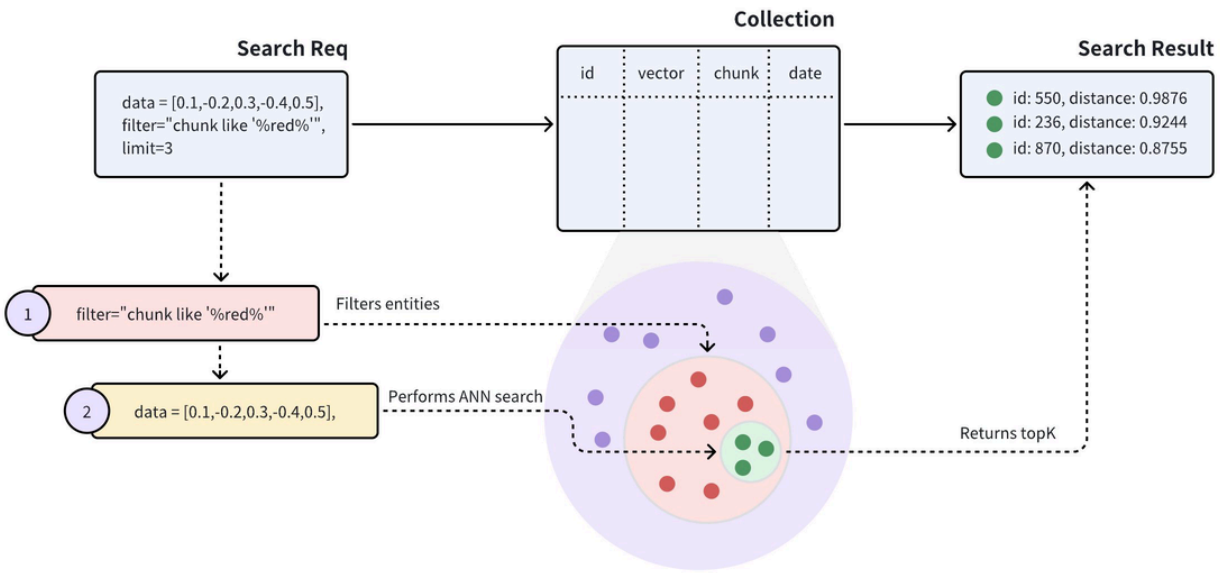
```python
def cross_encoder_hybrid_search(query: str):
    # 初始化一个cross-encoder的reranker
    cross_encoder_ranker = create_cross_encoder_ranker([query])
    # 混合检索
    hybrid_search(query, cross_encoder_ranker)


cross_encoder_hybrid_search("教育的负面作用是什么")
```

6.5 过滤

除了向量搜索，Milvus也支持普通字段过滤。Milvus 收到携带过滤条件的搜索请求后，会将搜索范围限制在符合指定过滤条件的实体内，再进行向量检索，从而提高查询效率。

如图：



Milvus中的混合检索条件就是一个普通字符串，其语法与MySQL的where条件几乎一样：

代码块

```python
filter = 'id == 1'

filter = 'id >= 1'

filter = 'title like "%重疾险%"'

filter = 'id in [11, 20, 30]'
```

具体语法参考官方文档：https://milvus.io/docs/zh/basic-operators.md

示例代码：

代码块

```
# RRF 混合检索
from pymilvus import AnnSearchRequest

def hybrid_search(query: str, ranker, filter_query: str = None):
    # 1.创建稠密、稀疏请求
    # 稠密
    query_dense_vector = embeddings.embed_query(query)
    dense_request = AnnSearchRequest(
        data=[query_dense_vector],
        anns_field="dense",
        param={"metric_type": "COSINE"},
        limit=3,
        filter=filter_query
    )
    # 稀疏
    sparse_request = AnnSearchRequest(
        data=[query],
        anns_field="sparse",
        param={"metric_type": "BM25"},
        limit=3,
        filter=filter_query
    )
    # 混合在一起
    reqs = [dense_request, sparse_request]

    # 2.设定融合策略，发送请求
    res = client.hybrid_search(
        collection_name=collection_name,
        reqs=reqs,
        ranker=ranker,
        limit=2,
        output_fields=['content', 'h2']
    )
    for hits in res:
        print("TopK results:")
        for hit in hits:
            print(hit)


```

```
40  from pymilvus import WeightedRanker
41
42  # 定义加权融合策略
43  ranker = WeightedRanker(0.5, 0.5)
44  # 测试
45  hybrid_search("教育的负面作用是什么", ranker, filter_query="h2 == '第一节 教育的功
    能'")
```

数组类型字段比较特殊，过滤条件语法也很特殊，详见文档：
https://milvus.io/docs/zh/array_data_type.md

7. LangChain-Milvus

在之前我们都是使用Milvus的官方客户端，它的功能更全面，可以支持灵活定制，但使用起来有点麻烦。比较适合于需要各种定制化功能的场景。

其实LangChain也是支持Milvus的，相比官方的客户端，LangChain功能有一定的限制，比如：

- 混合检索仅支持稀疏+稠密两种模式
- 不支持多模态向量检索
- 不支持对普通字段加索引
- ...

不过，这对于大多数的场景已经够用了。接下来我们就来学习LangChain的Milvus客户端。

7.1 准备工作

正式开始之前，我们先做一些准备工作。

7.1.1 安装依赖

LangChain提供了Milvus的特有依赖，需要单独安装：

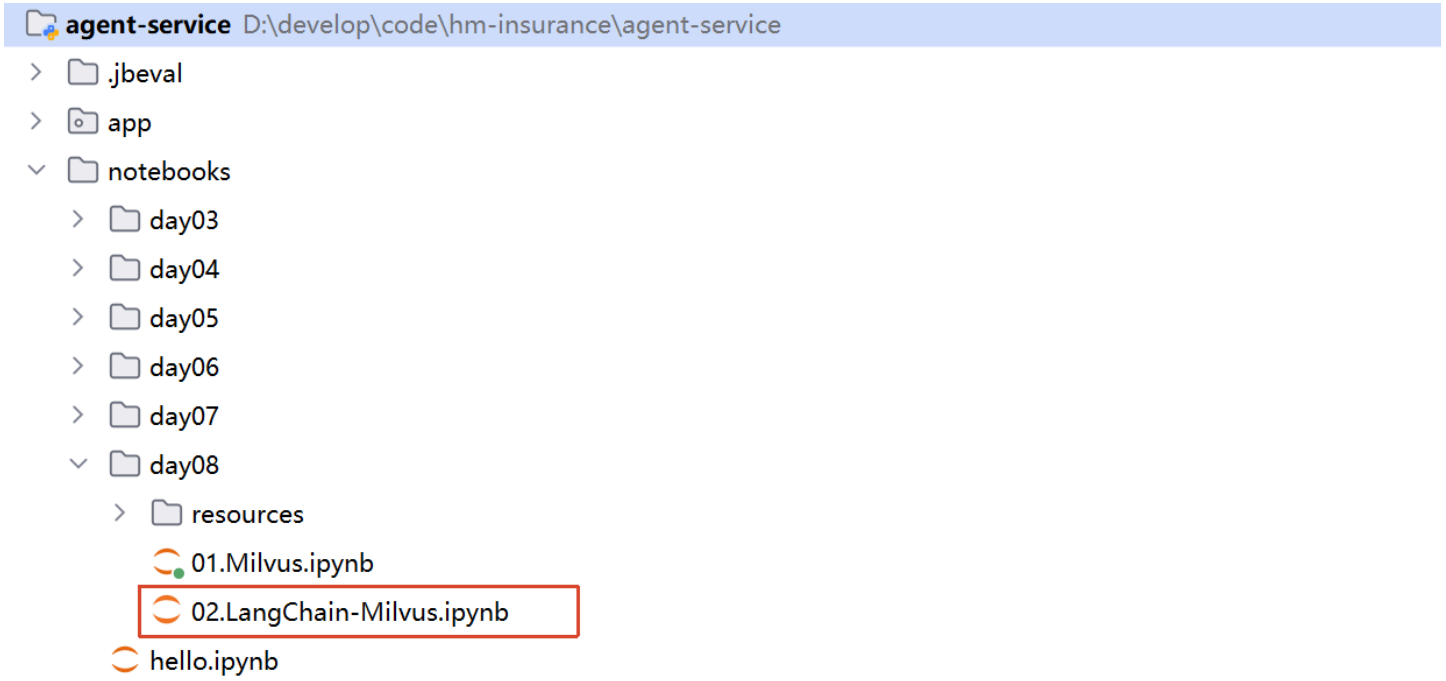
```
代码块
```

```
uv add langchain-milvus
```

7.1.2 准备文档

为了不与之前的Milvus测试冲突，我们会重新准备文档，创建新的Collection.

首先，在 `notebooks/day08` 下新建一个 `02.LangChain-Milvus.ipynb` 文件：



在第一个Cell中读取文档：

代码块

```python
# 一、准备文档
from langchain_text_splitters import MarkdownHeaderTextSplitter


def prepare_docs():
    # =============1.准备文档===============
    with open('./resources/评估.md', 'r', encoding='utf-8') as f:
        doc = f.read()

    # =============2.切分文档===============
    # 切分依据，这里是按照三级标题
    headers_to_split_on = [
        ("#", "h1"),
        ("##", "h2"),
        ("###", "h3"),
```

```
    ]

    # 创建切分器
    markdown_splitter = MarkdownHeaderTextSplitter(
        headers_to_split_on,
        strip_headers=False  # 在正文中是否去除标题，False，不去除
    )

    # 切分文档，返回的是 list[Document]
    docs = markdown_splitter.split_text(doc)

    # 为了方便查看，给每个Document加一个id，从1开始递增，但要注意的是LangChain的文档id要求是str
    for i, doc in enumerate(docs, start=1):
        doc.id = str(i)
    return docs

docs = prepare_docs()
```

⚡ **注意：**

- LangChain写入Milvus的文档如果自定义ID，则ID必须是字符串类型！
- 也可以不指定ID，则ID由Milvus生成，为INT64类型

7.1.3 向量模型

与之前类似，我们还是使用Ollama的向量模型：

代码块

```
# 二、初始化向量模型
from langchain_ollama import OllamaEmbeddings

# 1.初始化向量模型
embeddings = OllamaEmbeddings(
    model="qwen3-embedding:0.6b",  # 性价比高的模型
    dimensions=1024  # 可选：减少维度以节省存储
)
```

OK，准备工作都完成了。

7.2 VectorStore

LangChain中的操作是基于VectorStore的，当我们初始化VectorStore时，它可以自动帮我完成下列事情：

- 自动根据 `Document` 中的数据生成schema，有一些固定的字段：
  - `pk`：也就是主键，如果你提供了id，就用你的id值，没有的话会由Milvus自动生成
  - `text`：就是原始文本字段，存储的是 `Document` 的 `page_content` 值
  - `向量字段`：会根据 `text` 值自动生成稠密和稀疏向量，名字可自定义
  - `普通字段`： `Document` 的 `metadata` 会被展开，其中的每个字段都会成为Collection中的一个字段
- 自动给向量字段加索引，但其它字段不会有索引
- 初始化Collection
- 批量导入文档，也就是Entity

7.2.1 初始化VectorStore

示例代码：

代码块

```
# 三、初始化Collection，或者叫VectorStore
from langchain_milvus import Milvus, BM25BuiltInFunction
from app.core.config import settings

vectorstore = Milvus(
    embeddings,                                 # 稠密向量模型
    collection_name="langchain_collection", # collection名称
    builtin_function=BM25BuiltInFunction(   # 生成稀疏向量的函数
        analyzer_params={"type": "chinese"} # 指定中文分词
    ),
    vector_field=["dense", "sparse"],       # 向量字段，包括稠密和稀疏
    connection_args={
        "uri": settings.rag.milvus_url,     # milvus的uri路径
    },
```

```
    # index_params=[index_param],                # 可选，索引参数，必须与vector_field对应
    drop_old=True,                               # 是否删除旧的collection，避免重复创建
)
```

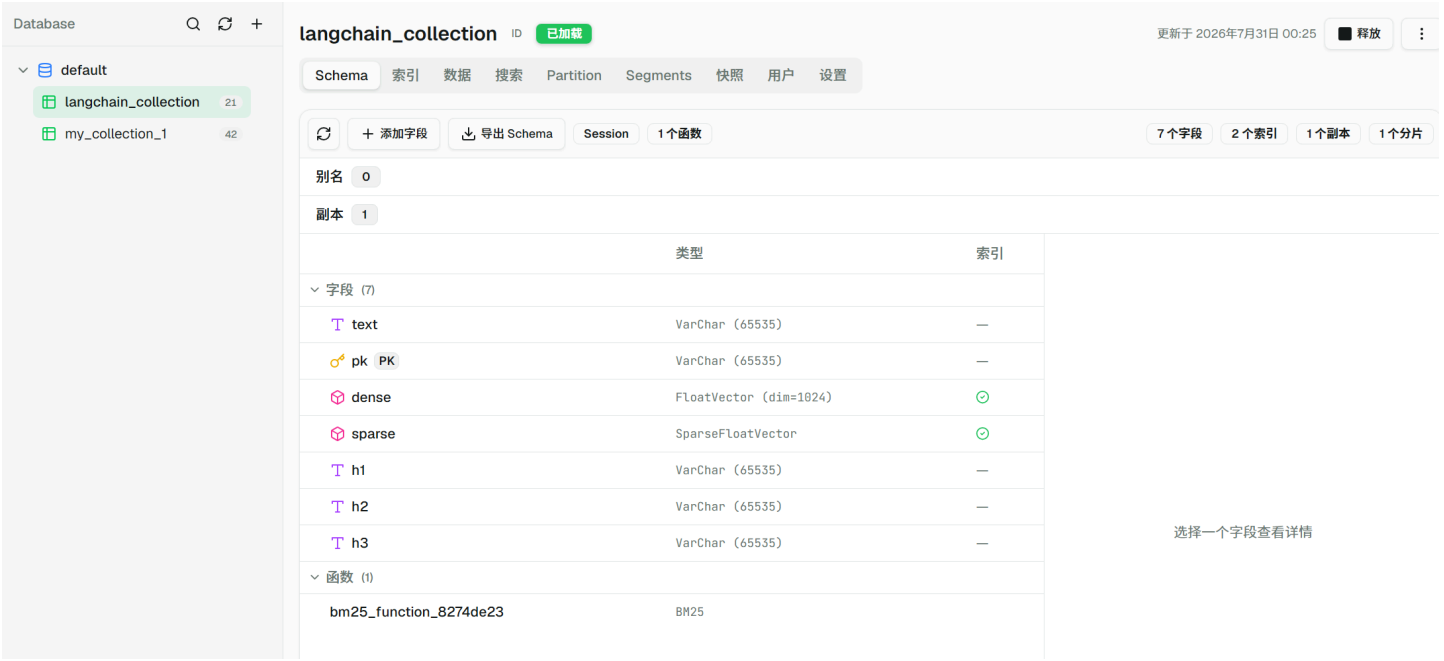
7.2.2 批量新增

新增调用 `add_documents()` 直接 传入 `list[Document]` 即可，LangChain会自动根据Document生成Schema、Index、Collection，完全不用我们操心。

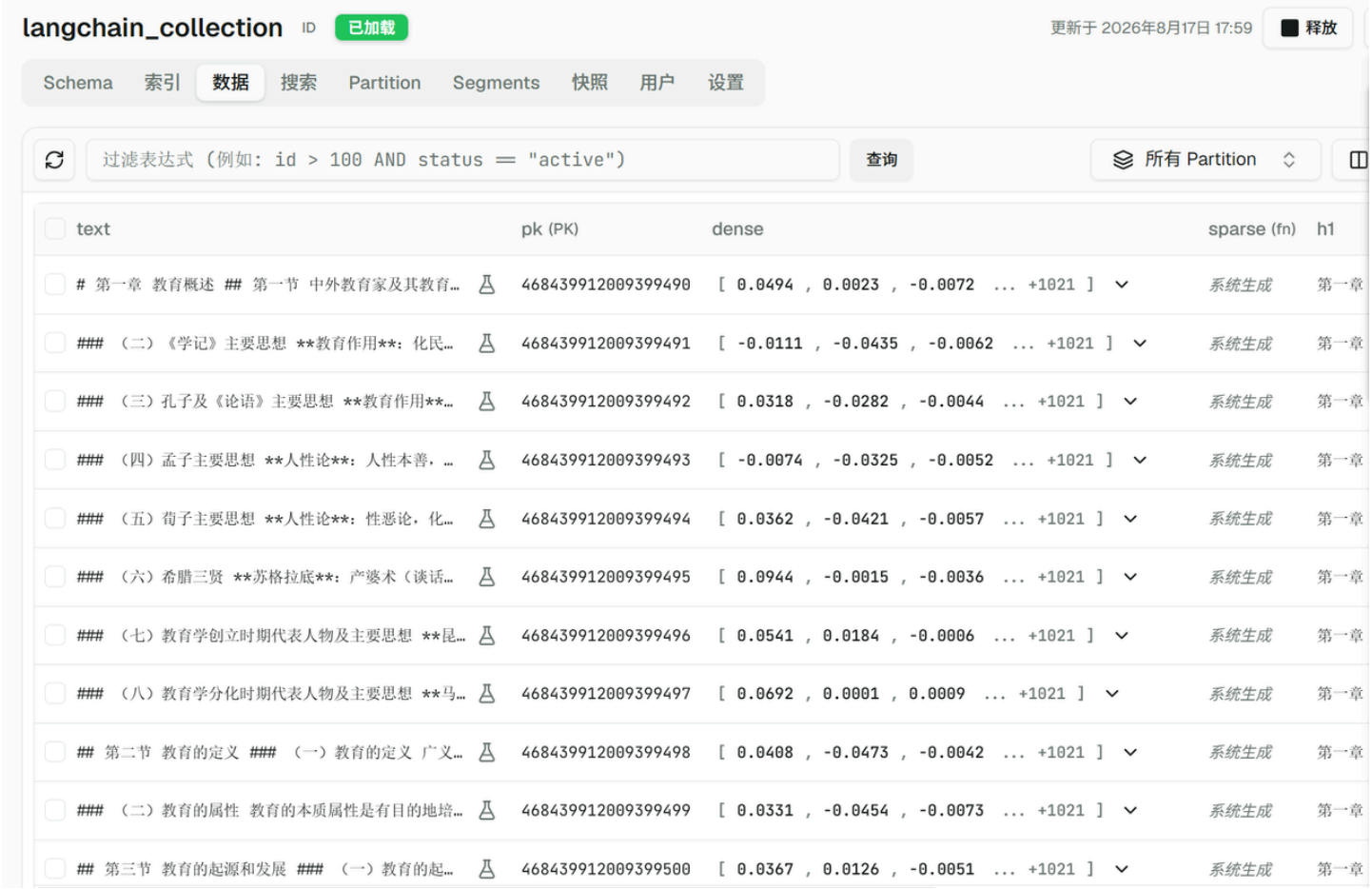
代码块

```
# 四、添加数据
vectorstore.add_documents(docs)
```

此时，查看GUI界面，会发现Collection创建完毕了：



而且数据页已经导入：



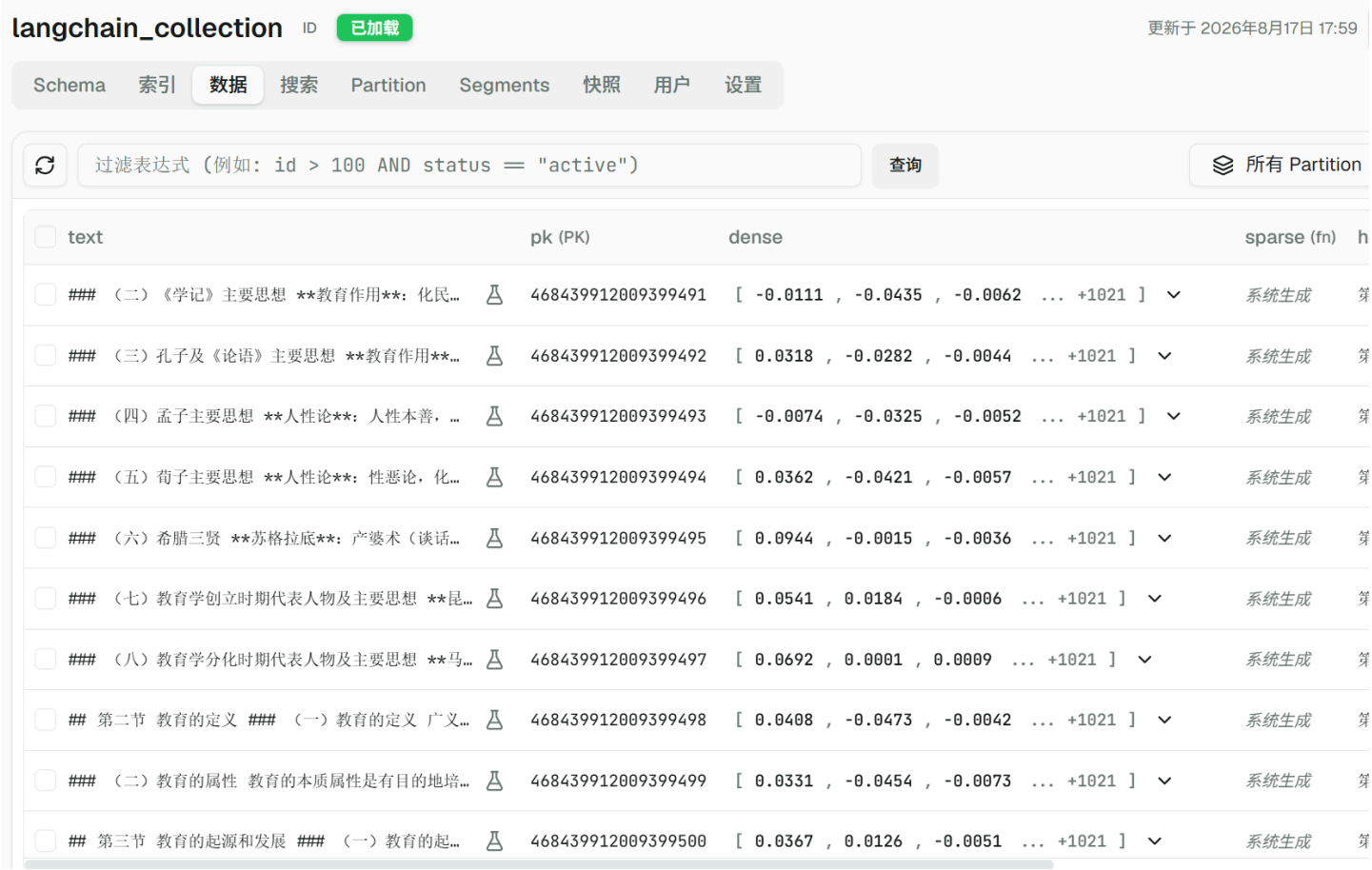
7.2.3 删除

支持基于id的删除：

代码块

```
# 1.删除，传入id集合
vectorstore.delete([468439912009399490])
```

查看GUI界面，可以发现id为 `468439912009399490` 的文档已经没有了，总数量只有20了：



7.3 检索

LangChain-Milvus实现了VectorStore接口，所以用法与昨天讲的VectorStore基本一样，并且也支持混合检索和融合策略了。

我们以 `similarity_search_with_score` 为例，它接收参数：

- `query`：查询条件
- `k`：查询文档数量
- `ranker_type`：重排策略，目前只支持 `weighted`、`rrf` 两个值
- `ranker_params`：重排参数，`ranker_type` 不同，传入值也不同
  - `weighted`：传入 `{"weights": [0.4, 0.6]}`
  - `rrf`：传入 `{"k": 60}`

⚡ **注意**：一旦你的Collection设定了稀疏向量，LangChain默认会自动开启**加权混合检索**，必须

传入混合检索参数，否则会报错。

7.3.1 基于权重的混合检索

代码块

```
# 五、基于权重的混合检索
query = "给教师的建议是谁写的"

# 带得分的相似度检索，返回值是list[tuple[Document, float]]
result = vectorstore.similarity_search_with_score(
    query,
    ranker_type="weighted",
    ranker_params={"weights": [0.4, 0.6]}
)

for doc, score in result:
    print(f"============score: {score}==============")
    print(doc.model_dump_json(indent=2))
```

7.3.2 基于RRF的混合检索

代码块

```
# 六、基于RRF的混合检索
query = "给教师的建议是谁写的"

# 带得分的相似度检索，返回值是list[tuple[Document, float]]
result = vectorstore.similarity_search_with_score(
    query,
    ranker_type="rrf",
    ranker_params={"k": 60}
)

for doc, score in result:
    print(f"============score: {score}==============")
    print(doc.model_dump_json(indent=2))
```

7.3.3 过滤搜索

LangChain是支持过滤检索的，表达式与Milvus一样：

代码块

```
query = "给教师的建议是谁写的"

# 带得分的相似度检索，返回值是list[tuple[Document, float]]
result = vectorstore.similarity_search_with_score(
    query,
    ranker_type="rrf",
    ranker_params={"k": 60},
    expr='h3 like "%代表人物%"'
)

for doc, score in result:
    print(f"============score: {score}==============")
    print(doc.model_dump_json(indent=2))
```

7.3.4 基于Cross-Encoder的混合检索

rrf和加权是LangChain默认支持的重排方案，因此可以用 `ranker_type` 直接指定。

但LangChain是不支持Cross-Encoder混合检索的，此时不能用 `ranker_type` 了，而是直接传递一个自定义的 `reranker` .

例如，我们要使用DashScope的重排模型，这么做：

1. 先定义生成ranker的函数

代码块

```

def create_cross_encoder_ranker(queries: list[str]):
    return Function(
        name="dashscope_semantic_ranker",  # ranker名词，唯一即可
        input_field_names=["text"],  # 原始文档字段
        function_type=FunctionType.RERANK,  # ranker类型，这里是固定值
        params={
```

```
            "reranker": "model",  # rerank类型，基于模型rerank，也就是cross-encoder
            "provider": "ali",  # rerank模型提供者
            "model_name": "gte-rerank-v2",  # rerank模型名称
            "queries": queries,  # 查询条件
            "max_client_batch_size": 5,  # 向模型发送请求时的批处理限制
        },
    )
```

2. 再定义混合检索函数：

代码块

```
def cross_encoder_hybrid_search(query: str, filter: str = ""):
    # 1.混合搜索
    _search_results = vectorstore.similarity_search_with_score(
        query,
        k=3,              # 最终返回的TopK
        fetch_k=5,        # 初筛时召回的文档的数量
        expr=filter,
        reranker=create_cross_encoder_ranker([query])
    )
    return _search_results
```

测试：

代码块

```
query = "教育的负面作用是什么"
results = cross_encoder_hybrid_search(query, 'h2 like "%教育的功能%"')
```

结果：

代码块

```
============score: 0.29470494389533997==============
{
  "id": null,
  "metadata": {
    "h1": "第二章 教育基本原理",
    "h2": "第一节 教育的功能",
    "h3": "（一）个体发展功能和社会发展功能",
```

```
    "pk": "15"
  },
  "page_content": "# 第二章 教育基本原理 \n## 第一节 教育的功能 \n### （一）个体发展功能和社会发展功能 \n教育的正向功能（积极功能）指教育有助于社会进步和个体发展的积极影响和作用。 \n教育的负向功能（消极功能）指阻碍社会进步和个体发展的消极影响和作用。 \n教育的显性功能是指教育活动依照教育目的，在实际运行中所出现的与之相吻合的结果。 \n教育的隐性功能指伴随显性功能所出现的非预期性的功能。",
  "type": "Document"
}
```